

Testing and Quality Assurance

Francesco Guerra
francesco.guerra@unimore.it

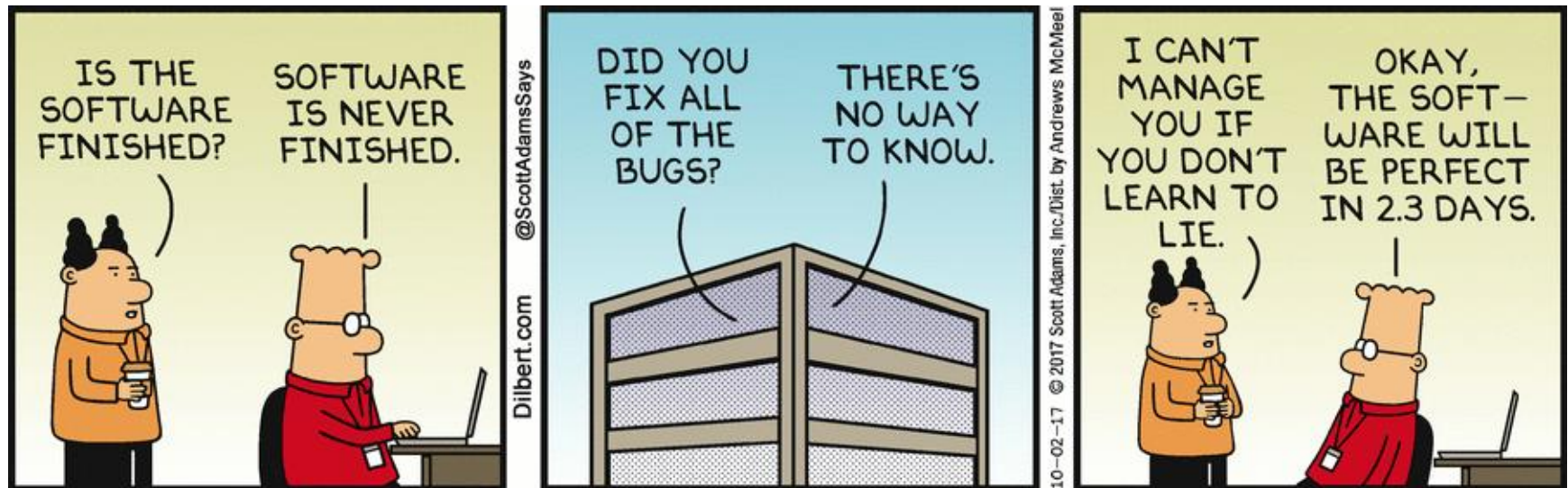


UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Objectives

- ▶ Understand the basic techniques for software verification and validation and when to apply them.
- ▶ Analyze the basics of software testing along with a variety of software testing techniques.

Software Is Never Finished



<http://dilbert.com/strip/2017-10-02>

Introduction to Testing and Quality Assurance

- ▶ The ultimate goal of software development is to **produce high-quality software**: reliable, maintainable, and aligned with user needs.
- ▶ **Testing** involves techniques for **detecting errors** in a software product.
 - ▶ It is a **verification activity**, ensuring the product behaves as expected.
 - ▶ Testing helps confirm that quality goals are being met but **does not create quality by itself**.
- ▶ **Quality Assurance (QA)** includes all **activities for measuring and improving quality**.
 - ▶ It covers the **entire development process**, from planning and design to testing and deployment.
 - ▶ QA also includes **training, process definition, and preparation** of the development team.
 - ▶ Its goal is to **prevent defects** before they occur.
- ▶ **Quality Control (QC)** focuses on **verifying the product's quality**.
 - ▶ It involves **detecting and correcting defects** before release.
 - ▶ QC ensures that the final software meets its **defined quality standards**.

Introduction to Testing and Quality Assurance

- ▶ **Static Structure vs. Dynamic Behavior**
 - ▶ A **software program** has both a **static structure** (its code and organization) and a **dynamic behavior** (how it executes and interacts at runtime).
 - ▶ **Intermediate software products**—such as requirements, analysis, and design documents—have only a **static structure**, not dynamic behavior.

Main Techniques for Error Detection

▶ Testing

- ▶ The process of **designing test cases**, **executing the program** in a controlled environment and **verifying** that the output is correct.
- ▶ Focuses on **dynamic behavior** and detects faults during execution.

▶ Inspections and Reviews

- ▶ Applied to both **code** and **intermediate software artifacts**.
- ▶ Involve **peer review**, requiring participation from **multiple people** besides the original author.
- ▶ Aim to **find defects early** by human examination of documents or code.

▶ Formal Methods

- ▶ Use **mathematical techniques** to *prove* that a program is correct.
- ▶ Provide strong guarantees but are **rarely used** in commercial or business contexts due to their **complexity and cost**.

▶ Static Analysis

- ▶ Analyzes the **static structure** of code or documents **without execution**.
- ▶ Usually **automated**, identifying **errors or error-prone patterns** early in development.

Two Perspectives on Quality

- ▶ The quality of any software product can be defined in **two ways**:
 - ▶ It conforms to specifications.
 - ▶ It serves its intended purpose.
- ▶ A product may **perfectly conform** to its specifications yet **fail to be useful** if those specifications do not reflect real user needs.
- ▶ **Verification**
 - ▶ The act of checking that the software **conforms to its requirements and specifications**.
 - ▶ Answers the question: *“Are we building the product right?”*
- ▶ **Validation**
 - ▶ The act of checking that the **finished software** meets the **users’ real needs and expectations**.
 - ▶ Answers the question: *“Are we building the right product?”*
 - ▶ Usually performed **at the end of the project**, once the full system is available.

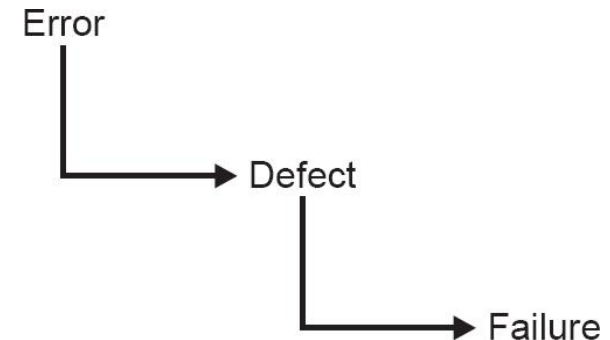
Validation vs Verification

Verification	Validation
Verification addresses the concern: "Are you building it right?"	Validation addresses the concern: "Are you building the right thing?"
Ensures that the software system meets all the functionality.	Ensures that the functionalities meet the intended behavior.
Verification takes place first and includes the checking for documentation, code, etc.	Validation occurs after verification and mainly involves the checking of the overall product.
Done by developers.	Done by testers.
It has static activities, as it includes collecting reviews, walkthroughs, and inspections to verify a software.	It has dynamic activities, as it includes executing the software against the requirements.
It is an objective process and no subjective decision should be needed to verify a software.	It is a subjective process and involves subjective decisions on how well a software works.



Why software fails?

- ▶ People make mistakes because they are **human and fallible**, but certain **pressures and conditions** make errors **more likely**.
- ▶ **Common Sources of Error**
 - ▶ **Tight deadlines** and schedule pressure.
 - ▶ **System and organizational complexity**, making interactions hard to predict.
 - ▶ **Constantly changing technology**, requiring adaptation and learning.
- ▶ These factors increase the likelihood of **defects** in
 - ▶ **Specifications** (what should be built),
 - ▶ **Designs** (how it should be built), and
 - ▶ **Code** (the implementation itself).
- ▶ **From Error to Failure**
 - ▶ An **error (or mistake)** introduced by a person leads to a **defect (or fault)** in the product, which can eventually cause an **observed failure** during execution or use.
 - ▶ Therefore, **detecting and preventing errors early** is key to avoiding **major system failures** later.



Why software fails?

▶ Fault (Defect or Bug)

- ▶ A **fault** is a **condition** in the **system** that may cause a **failure**.
- ▶ It is introduced by an **error** made by a software engineer.
- ▶ A fault is also called a **bug**.
- ▶ **Example:** A **requirement** that is **hard to understand** represents a **fault** in the specification.

▶ Failure (Problem)

- ▶ A **failure** is the **inability of the system** to perform a function according to its **specification**.
- ▶ It is the **observable result** of one or more **defects** in the system.
- ▶ **Example:** An **unclear requirement** leads to a **design or coding mistake**, which **manifests as a software failure** during execution.

▶ Error-Fault-Failure Relationship

- ▶ An **error** (human mistake) introduces a **fault**.
- ▶ The **fault** may cause a **failure** if the faulty part of the code is executed.
- ▶ Not all defects cause failures: some remain **dormant**, because the relevant logic is **never executed**.

Why software fails?

- ▶ Systems may fail not only because of software defects, but also due to **environmental conditions** such as **radiation, magnetism, electrical interference, or pollution.**
- ▶ To avoid system failure, we must:
 - ▶ **Avoid introducing errors and faults, and**
 - ▶ **Detect and correct those that occur.**
- ▶ **Testing** plays a central role in both prevention and correction:
 - ▶ **Begin testing as early as possible**
 - ▶ from the moment development starts.
 - ▶ **Continue testing throughout the entire process, until you are confident that no serious system failures remain.**

Keeping Software under Control

- ▶ The **quality of software** is difficult to define,
 - ▶ one key measure is whether the system **meets its users' requirements**.
- ▶ Testing helps ensure that **functional** and **non-functional requirements** are **examined and verified** before the system goes into service.
- ▶ Any **defects identified** during testing are **reported to the development team** for correction.
 - ▶ Through **systematic coverage**, testing provides a way to measure certain **aspects of quality**.
- ▶ Testing is a **core activity of quality assurance**,
 - ▶ helping to ensure that systems are deployed **without critical defects** that could cause failure.
- ▶ Testing itself **does not remove defects**,
 - ▶ by **detecting and reporting them**, it makes their **removal possible**, thus contributing to **higher system quality**.

Keeping Software under Control

- ▶ You must also distinguish between
 - ▶ the severity of a fault, the impact or consequences it may have
 - ▶ the priority, a measure of its importance.
- ▶ Software failures could have high severity and low priority and vice versa.
 - ▶ A bug causing a program crash under rare conditions will usually be given a lower priority than less severe bugs that occur under most common conditions.

Testing

- ▶ Testing can be defined as
 - ▶ A process of analyzing a software item to detect the differences between existing and required conditions (that is defects/errors/bugs) and to evaluate the features of the software item (ANSI/IEEE 1059 standard - inactive).
 - ▶ Testing is an activity performed for evaluating product quality, and for improving it, by identifying defects and problems.
(Guide to Software Engineering Body of Knowledge - 2004)

Testing

- ▶ **Testing** is the **systematic and methodical examination** of a *work product* using a **variety of techniques**, with the specific goal of showing that the product **fulfils its intended purpose**.
- ▶ It is performed in an environment that **closely resembles real operation**, ensuring that observed behaviour reflects what users will experience in production.
- ▶ **Key Characteristics**
 - ▶ **Systematic** - testing must be **planned** and organized, not random.
 - ▶ **Methodical** - it follows a **defined process** with clear steps and objectives.
 - ▶ **Work product** - testing applies not only to **code**, but also to **requirements, design documents, and specifications**.
 - ▶ **Variety of techniques** - includes **reviews, static analysis, dynamic testing, simulations, etc.**

What Does Testing Aim to Do?

- ▶ Check that all **requirements** have been satisfied.
- ▶ Verify that the item under test is **complete** and behaves as **users and stakeholders expect**.
- ▶ Instil confidence in the **quality** of the item under test.
- ▶ Find failures and defects early, to prevent them from reaching production.
- ▶ Provide information to support **decision-making**, e.g., whether the software is ready for release.
- ▶ Reduce the risk associated with inadequate software quality.
- ▶ Ensure compliance with contractual, legal, or standard requirements
 - ▶ for example, demonstrating **80% decision coverage** through testing.

▶ .

The Purposes of Testing

▶ Main Purposes

- ▶ To find defects in the software so they can be corrected or mitigated.
- ▶ To assess overall quality, providing assurance that the product
 - ▶ works for many considered cases, and
 - ▶ offers an estimate of remaining faults.

▶ Limits of Testing

- ▶ Testing cannot prove that a product always works.
- ▶ It only shows that the product works for the specific cases tested.
- ▶ Correct testing provides reasonable confidence that the software works in similar situations, but not in all possible cases.
- ▶ Test results can verify whether quality goals have been met.
- ▶ Based on these results, further testing may be recommended.

Testing Techniques

- ▶ Who does the testing?
 - ▶ **Programmers** design and execute test cases to **verify their own code**.
 - ▶ This activity is typically referred to as **unit testing**.
 - ▶ The goal is to ensure that **individual components** function correctly before integration.
 - ▶ **Testers** are technical professionals who **design, implement, and execute** test cases.
 - ▶ Some testers also **analyze test results statistically** to **assess product quality**.
 - ▶ They often **support release decisions** by providing data on residual risk and defect levels.
 - ▶ **Users** participate in testing to identify **usability issues** and to expose the software to **real-world usage scenarios**.
 - ▶ Users may also take part in **acceptance testing**:
 - **Alpha testing** - performed by users **within** the developing organization.
 - **Beta testing** - performed by users **outside** the organization.

Testing Techniques

▶ When to Start Testing?

- ▶ **Start testing as early as possible** in the software development process.
- ▶ An **early start** reduces **costs and rework** and helps deliver **error-free software** to the client.
- ▶ Testing should begin in the **Requirements Gathering** phase and continue until **deployment**.
- ▶ Testing takes **different forms** at every stage of the **Software Development Life Cycle (SDLC)**:
 - ▶ **Requirements phase**: analyzing and verifying requirements.
 - ▶ **Design phase**: reviewing and improving the software design.
 - ▶ **Implementation phase**: developers test completed code (unit testing).
- ▶ Testing is thus a **continuous activity**, not a final step.

Static testing and dynamic testing

▶ Static and Dynamic Testing

- ▶ **Static Testing** refers to **testing without executing the code**.
 - ▶ Detects **defects early**, before they turn into failures.
 - ▶ Failures originate from **human errors** – wrong assumptions or mistakes when creating documents such as **requirements or specifications**.
 - ▶ **Early testing** is crucial because **errors are cheaper to fix** than defects or failures.
- ▶ Hence, testing should **start as early as possible** in the development process.

▶ Dynamic Testing

- ▶ Involves **executing the program** with **test data** to verify its behavior.
- ▶ It focuses on **test execution**, observing how the system performs under various conditions.
- ▶ Both **static and dynamic testing** are essential components of **software testing** as a discipline.

Testing Techniques

- ▶ What is tested?
 - ▶ Functional Testing
 - ▶ Based on the specifications of the software that is to be tested.
 - ▶ Non-functional Testing
 - ▶ Non-functional testing involves testing a software from the requirements which are nonfunctional in nature but important such as performance, security, user interface, etc.
 - ▶ White-box testing
 - ▶ The focus is on the internal structure of the system.
 - ▶ Regression test
 - ▶ Testing after code has been changed

Functional testing

- ▶ It looks at the specific functionality of a system.
 - ▶ This includes checks for completeness, correctness and appropriateness.
- ▶ Functional testing is carried out at all levels of testing, from unit through to acceptance testing
- ▶ Functional testing is called specification-based testing or black-box testing.
 - ▶ It can be measured in terms of the percentage of requirements covered by the tests.
- ▶ Designing tests at this level often requires specific domain skills.

Non-Functional Testing

- ▶ Focuses on the **behavioural and quality attributes** of the system, such as **usability, performance, efficiency, and security**.
- ▶ Should be **performed at all levels** of testing to detect potential defects as **early as possible**.
- ▶ Often uses **black-box testing techniques**, for example:
 - ▶ Checking that a **flight can be booked within a specific time frame**.
- ▶ Typically requires **specialist knowledge and skills**, such as:
 - ▶ Understanding **performance testing** methods,
 - ▶ Recognizing **technology-specific weaknesses**.
- ▶ **Results can be measured**, for instance:
 - ▶ Verifying the **percentage of mobile devices** successfully tested for compatibility.
- ▶ Non-functional testing can be mapped to quality models such as **ISO/IEC 25010 - Systems and Software Quality Requirements and Evaluation (SQuaRE)**

Non-Functional Testing

- ▶ **Purpose:** to identify **bottlenecks** or **performance issues** in software systems.
 - ▶ Network delay
 - ▶ Client-side processing
 - ▶ Database transaction processing
 - ▶ Load balancing between servers
 - ▶ Data rendering on the client side
- ▶ **Key Aspects of Performance Testing**
 - ▶ **Speed** - response time, data rendering, and data access.
 - ▶ **Capacity** - the amount of workload the system can handle.
 - ▶ **Stability** - consistent performance under expected loads.
 - ▶ **Scalability** - ability to maintain performance as load increases.
- ▶ Performance testing may be **qualitative** (observing user experience) or **quantitative** (measuring metrics).
- ▶ It includes subtypes such as:
 - ▶ **Load testing** - measuring performance under expected conditions.
 - ▶ **Stress testing** - evaluating performance under extreme conditions or peak loads.

Non-Functional Testing

▶ Load Testing

- ▶ **Purpose:** to test the **behavior of software under load**, by simulating access to large volumes of data or many simultaneous users.
- ▶ Can be executed under both **normal** and **peak load** conditions.
- ▶ Identifies the **maximum capacity** of the software and how it behaves during **high-usage periods**.

▶ Execution

- ▶ Usually performed using **automated tools** such as: **LoadRunner, AppLoader, IBM Rational Performance Tester, Apache JMeter, Silk Performer, Visual Studio Load Test**, etc.
 - ▶ In these tools, **virtual users (VUsers)** are created to simulate real user activity.
 - ▶ The **number of users** can be **increased or decreased**, either **concurrently or incrementally**, depending on the test objectives.

Non-Functional Testing

▶ Stress Testing

- ▶ **Purpose:** to evaluate the **behavior of software under abnormal or extreme conditions.**
- ▶ Involves applying a **load beyond the system's expected capacity** or **removing critical resources.**
- ▶ The objective is to identify the **breaking point** of the system — how and when it fails, and how gracefully it recovers.

▶ Typical Stress Scenarios

- ▶ Random **shutdown or restart of network ports.**
- ▶ Turning the **database** on or off unexpectedly.
- ▶ Running additional **processes** that consume CPU, memory, or server resources.
- ▶ Forcing **resource exhaustion** (e.g., disk space, connections, threads).

Non-Functional Testing

▶ Usability Testing

- ▶ **Purpose:** to identify **errors and potential improvements** in software by **observing users** as they interact with the system.
- ▶ **Usability** can be described in terms of **five key factors**:
 - ▶ **Efficiency of use** - how quickly users can complete tasks.
 - ▶ **Learnability** - how easily new users can learn to use the system.
 - ▶ **Memorability** - how easily returning users can reestablish proficiency.
 - ▶ **Errors/Safety** - how easily users can recover from mistakes.
 - ▶ **Satisfaction** - users' comfort and positive attitude toward the system.
- ▶ Usability can be **measured as the outcome of user interaction** with the system.
- ▶ The goal is achieved when **users can accomplish intended tasks effectively and efficiently** using available resources.
- ▶ Usability is formally defined in **international standards** such as **ISO 9126, ISO 9241-11, ISO 13407, and IEEE Std. 610.12.**

Non-Functional Testing

▶ Security Testing

- ▶ **Purpose:** to identify **weaknesses, and vulnerabilities** in software from a **security and risk** point of view, ensuring that
 - ▶ The **software is protected** against known and unknown threats.
 - ▶ **Data remains secure** during storage, transmission, and processing.
 - ▶ The system complies with **security regulations and standards**.
- ▶ **Key Security Principles**
 - ▶ **Confidentiality** - preventing unauthorized access to data.
 - ▶ **Integrity** - ensuring that information is accurate and unaltered.
 - ▶ **Authentication** - verifying the identity of users or systems.
 - ▶ **Authorization** - ensuring access only to permitted functions or data.
 - ▶ **Availability** - keeping services accessible and operational.
 - ▶ **Non-repudiation** - preventing users from denying their actions.
- ▶ **Common Vulnerabilities to Test**
 - ▶ **Input checking and validation errors**
 - ▶ **SQL injection** and other **injection flaws**
 - ▶ **Weak session management** or improper logout handling
 - ▶ **Buffer overflows** and memory corruption issues

Non-Functional Testing

▶ Portability Testing

- ▶ **Purpose:** to ensure **software reusability** and that it can be moved and executed on different platforms.
- ▶ Involves activities such as:
 - ▶ **Transferring** installed software from one computer or environment to another.
 - ▶ **Building executables** (e.g., .exe) and verifying that they run correctly on multiple platforms.
- ▶ Portability testing is considered a **subtype of system testing**.
- ▶ **Focus Areas**
 - ▶ Computer hardware configurations
 - ▶ Operating systems
 - ▶ Web browsers and runtime environments
- ▶ **Preconditions**
 - ▶ The software must be **designed and coded** with **portability requirements** in mind.
 - ▶ **Unit testing** and **integration testing** should be **completed**.
 - ▶ A **stable test environment** must be **established** before execution.

Non-Functional Testing Other

▶ Conformance Testing

- ▶ Ensures that the **software product complies** with a defined set of **standards, regulations, or policies**.
- ▶ **Test cases** are derived directly from the **standards or specifications** the software must follow.
- ▶ Common in **regulated domains** such as healthcare, finance, telecommunications, or safety-critical systems.

▶ Configuration Testing

- ▶ Verifies that the software **functions correctly under different configurations**.
- ▶ Examples include:
 - ▶ Running the product with **multiple databases**.
 - ▶ Operating across **different networks or environments**.
- ▶ Helps identify **compatibility issues** and ensures consistent performance across setups.

▶ User Interface (UI) Testing

- ▶ Focuses exclusively on the **user interface** and **user interaction**.
- ▶ Checks for **consistency, responsiveness, accessibility**, and compliance with **design guidelines**.
- ▶ Ensures that the interface is **intuitive and user-friendly** across devices and screen sizes.

White-box testing

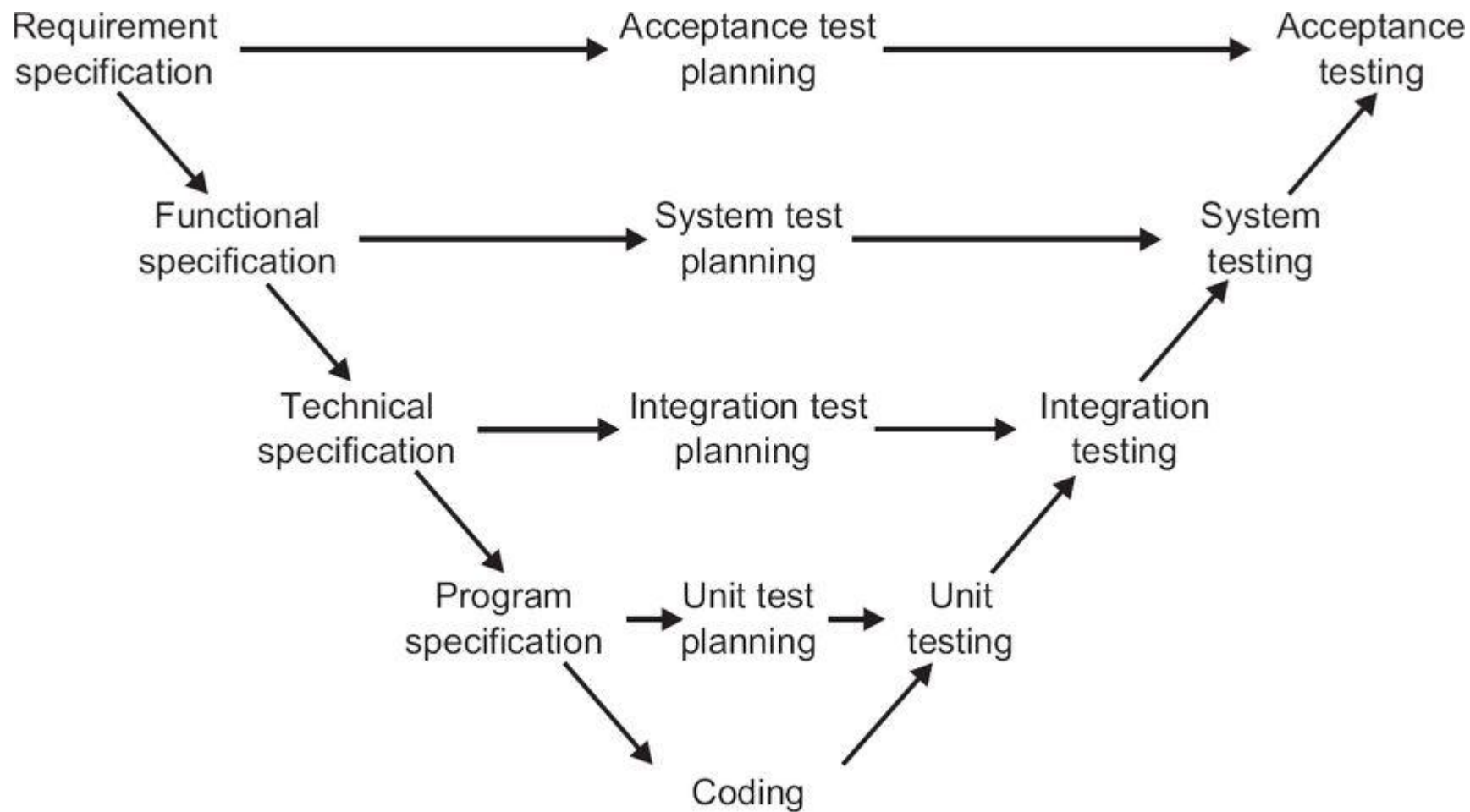
- ▶ In white-box testing our focus is on the internal structure of the system.
 - ▶ This could be the code itself, an architectural definition or data flows through the system.
- ▶ White-box testing is commonly carried out at unit and component integration test levels.
 - ▶ Here, common measures include code and interface coverage (percentage of code and interfaces exercised by tests).
- ▶ It can also be carried out at the higher levels of testing where a structural definition of the system exists.

Regression Testing

- ▶ Whenever a **change or fix** is made in a software application, there is a risk that **other areas** of the system have been **unintentionally affected**.
- ▶ **Regression testing** is performed to verify that:
 - ▶ A **fixed bug** has not caused a **new defect** elsewhere.
 - ▶ The change does not violate any existing **functionality or business rules**.
- ▶ The main goal is to ensure that **software modifications** do not **introduce new faults**.
- ▶ **Importance of Regression Testing**
 - ▶ **Minimizes testing gaps** when verifying applications after changes or updates.
 - ▶ **Confirms** that new or modified features **do not affect existing functionality**.
 - ▶ **Reduces risk** by ensuring system stability after each iteration or release.
 - ▶ **Improves test coverage** without compromising project **timelines** especially when automated regression suites are used.

Test Levels

- At which level the software is tested?



Test Levels

Component (Unit) Testing

- ▶ Software is typically written as **individual components or units**, also referred to as **modules** or **programs**.
- ▶ Components are **developed and tested in isolation** before being integrated into the larger system.
- ▶ **Component (unit) testing** focuses on verifying each individual unit independently.
- ▶ **Scope of Unit Testing**
 - ▶ **Functional requirements** - e.g., verifying that items can be removed from a shopping cart.
 - ▶ **Non-functional characteristics** - e.g., checking for **memory leaks** that may slow down the system.
 - ▶ **Structural testing** - assessing the **percentage of code coverage** achieved by tests.
- ▶ **Objectives of Unit Testing**
 - ▶ **Reduce risk** early in development.
 - ▶ **Verify** that both **functional and non-functional behaviours** match the design and specification.
 - ▶ **Build confidence** in the component's quality.
 - ▶ **Detect defects** in the component and **prevent them from propagating** to higher testing levels.

Test Levels

Component (Unit) Testing

- ▶ **Test Bases** can include:
 - ▶ Detailed design and component specifications.
 - ▶ Data models or other documents describing expected functionality.
 - ▶ The code itself, used as the basis for creating and executing unit tests.
- ▶ **Test Objects**
 - ▶ Code and data structures.
 - ▶ Classes, methods, and database modules.
 - ▶ Other individual pieces of code that can be tested independently.
- ▶ **Typical Defects and Failures**
 - ▶ Incorrect functionality due to logical errors.
 - ▶ Data flow problems, e.g., code requests an input but produces no output.
 - ▶ Overly complex code constructs that reduce maintainability and increase the risk of errors.
- ▶ **Approaches and Responsibilities**
 - ▶ Developers typically perform and fix issues found during unit testing.
 - ▶ A key method is **Test-Driven Development (TDD)**, from *eXtreme Programming*: Write test cases first, Then develop and modify the code until it passes all tests.
 - ▶ Automated regression testing is integrated into the process to detect issues quickly — a standard practice in iterative and agile development models.

Test Levels

Integration Testing

- ▶ After the individual **units** have been developed and tested, they are **combined to form larger subsystems or the complete system**.
- ▶ The purpose of integration testing is to **detect defects** in:
 - ▶ The **interfaces** between components, and
 - ▶ The **interactions** among integrated parts.
- ▶ Integration testing ensures that **units work together correctly** as a cohesive system.
- ▶ **Levels of Integration Testing**
 - ▶ **Component Integration Testing**
Focuses on the **interfaces and interactions** between integrated **components** within a single system.
 - ▶ **System Integration Testing**
Focuses on the **interfaces and interactions** between **different systems, packages, or microservices**. May also include **external interfaces** with third-party systems or organizations.
- ▶ **Objectives**
 - ▶ **Reduce risk** of integration failures.
 - ▶ **Verify** both **functional** and **non-functional** behavior of interfaces.
 - ▶ **Build confidence** in the **quality and reliability** of integrations.
 - ▶ **Identify defects** in interfaces or in the integrated components.
 - ▶ **Prevent defects** from propagating to higher testing levels.

Test Levels

Integration Testing

▶ Test Bases

- ▶ Integration testing is based on documentation and artifacts such as:
 - ▶ Software and system design documents
 - ▶ Use cases and workflows
 - ▶ Component or system architecture
 - ▶ External interface definitions

▶ Test Objects

- ▶ Subsystems and databases
- ▶ Infrastructure components
- ▶ Interfaces and APIs (Application Program Interfaces)
- ▶ Microservices and service interactions

▶ Typical Defects and Failures

- ▶ Incorrect or missing data, or incorrect data encoding
- ▶ Wrong sequencing or timing of interface calls
- ▶ Interface mismatches or communication failures between components
- ▶ Ignored or poorly handled error conditions in communication
- ▶ Incorrect assumptions about the meaning, units, or boundaries of exchanged data

Test Levels

System Integration Testing

▶ Typical Defects and Failures

- ▶ **Inconsistent message structures** between systems.
- ▶ **Incorrect or missing data** in exchanged messages.
- ▶ **Incorrect data encoding** (similar to component integration issues).
- ▶ **Interface mismatches** between interacting systems.
- ▶ **Communication failures** across networked systems.
- ▶ **Incorrect assumptions** about the **meaning, units, or boundaries** of exchanged data.
- ▶ **Non-compliance** with mandatory security or regulatory requirements.

▶ Approaches and Responsibilities

- ▶ **Component integration testing** is typically performed by **developers**, focusing on internal consistency and interface correctness.
- ▶ **System integration testing** may involve **testers, system engineers, and external partners**, especially when multiple systems or organizations are involved.
- ▶ **Regression testing** is often **automated**, ensuring that changes do not introduce new integration issues during repeated builds.

Test Levels

System Testing

- ▶ Lower-level tests (unit and integration) often use **test cases** that do not fully represent real **operating conditions**.
- ▶ As a result, these tests may **miss defects** caused by **system-wide interactions** or **environmental factors**.
- ▶ **System testing** addresses this by testing the **entire integrated system** in an environment that closely replicates **production conditions**.
- ▶ The results of system testing help **stakeholders** make **release decisions**, including verifying **compliance with legal, regulatory, or industry standards**.
- ▶ The required system behaviour is defined in the **functional and non-functional requirements**.
- ▶ **Specific Objectives**
 - ▶ **Reduce risk** before deployment.
 - ▶ **Verify** that both **functional** and **non-functional behaviours** meet design and specification.
 - ▶ **Validate** that the system is **complete** and works as expected.
 - ▶ **Build confidence** in the **overall quality** of the product.
 - ▶ **Identify defects** that emerge only at the system level.

Test Levels

System Testing

- ▶ **Test Bases**
 - ▶ System and software requirement specifications (functional and non-functional) / manuals
 - ▶ Risk analysis reports
 - ▶ Use cases and user stories
 - ▶ Models of system behaviour and state diagrams
- ▶ **Test Objects**
 - ▶ Applications and complete software systems
 - ▶ Hardware/software integrated systems
 - ▶ Operating systems and runtime environments
 - ▶ System configuration data, i.e., parameters that can be adjusted for specific contexts or needs
- ▶ **Typical Defects and Failures**
 - ▶ Incorrect calculations or data processing errors
 - ▶ Unexpected or incorrect system behaviour under specific conditions
 - ▶ Incorrect control flow or data flow within the system
- ▶ **Approaches and Responsibilities**
 - ▶ Testers should participate in static reviews of requirements and documentation to detect ambiguities and misunderstandings early.
 - ▶ The test environment should closely replicate the production environment.
 - ▶ System testing is typically performed by independent testers.
 - ▶ Automation is often employed to improve efficiency and repeatability.



Test Levels

Acceptance Testing

- ▶ The main purpose is to give **end users and stakeholders confidence** that the **system functions according to their expectations** and business needs.
- ▶ It is typically the **final level of testing** before release to production.
- ▶ The emphasis is on **validation** (building the right product), rather than finding defects.
- ▶ **Forms of Acceptance Testing**
 - ▶ **User Acceptance Testing (UAT)**
 - ▶ Performed by **user representatives** to confirm that the system meets **business requirements**.
 - ▶ May include **factory** or **site acceptance testing** depending on the context.
 - ▶ **Operational Acceptance Testing (OAT) / also called operational readiness testing.**
 - ▶ Ensures that **processes, procedures, and resources** are in place for operation and maintenance.
 - ▶ Covers:
 - Backup and recovery procedures / Installation, uninstallation, and upgrades / Performance and disaster recovery / User management and security / Data migration and maintenance activities
 - ▶ **Contract and Regulatory Acceptance Testing**
 - ▶ **Contractual acceptance** - when acceptance criteria are defined in a **contract**.
 - ▶ **Regulatory acceptance** - ensures compliance with **legal, safety, or governmental standards**.
 - ▶ **Alpha and Beta Testing**
 - ▶ **Alpha testing** - performed by internal users within the development organization.
 - ▶ **Beta testing** - performed by external users in real-world environments.



Test Levels

Acceptance Testing

- ▶ **Specific Objectives**
 - ▶ Establish confidence in the overall system quality.
 - ▶ Validate completeness and confirm the system works as expected.
 - ▶ Verify that functional and non-functional behaviours meet specifications.
 - ▶ **Note:** defect discovery is **not the main goal**, though any found defects must be **logged and resolved**.
- ▶ **Test Bases**
 - ▶ User, business, legal, regulatory, and system requirements
 - ▶ Use cases and business process descriptions
 - ▶ Installation and deployment procedures
 - ▶ Risk analysis reports
 - ▶ Backup, restore, and disaster recovery procedures
 - ▶ Security standards or regulations
 - ▶ Non-functional requirements
 - ▶ Operations and maintenance documentation
 - ▶ Performance targets and database packages

Test Levels

Acceptance Testing

- ▶ **Test Objects**
 - ▶ System under test and its configuration
 - ▶ Configuration and production data
 - ▶ Business processes in a fully integrated environment
 - ▶ Operational and maintenance procedures
 - ▶ Forms, reports, and interfaces
 - ▶ **Typical Defects and Failures**
 - ▶ System workflows do not meet business or user requirements.
 - ▶ Business rules are incorrectly implemented.
 - ▶ The system fails to satisfy contractual or regulatory requirements.
 - ▶ Non-functional failures, e.g.
 - ▶ Security vulnerabilities
 - ▶ Performance inefficiencies under high load
 - ▶ Incompatibility with supported platforms
 - ▶ **Approaches and Responsibilities**
 - ▶ Acceptance testing is typically performed by customers or end users, though project team members may assist.
 - ▶ It is often viewed as the final test level in a sequential development life cycle, confirming that the system is fit for operational use.
-

Testing Techniques

- ▶ What testing does?
 - ▶ Testing relies upon an understanding of what the system, application or utility is meant to achieve.

Test basis:	The body of knowledge used as the basis for test analysis and design.
Test condition:	An aspect of the test basis that is relevant in order to achieve specific test objectives.
Test case:	A set of preconditions, inputs, actions (where applicable), expected results and postconditions, developed based on test conditions.
Test procedure:	A sequence of test cases in execution order, and any associated actions that may be required to set up the initial pre-conditions and any wrap up activities post execution.

Testing Techniques

- ▶ How do we generate and choose test cases?
 - ▶ **Intuition-Based Testing**
 - ▶ Relies on **experience and intuition** rather than formal methods.
 - ▶ Common in **Alpha and Beta testing**, where users or support staff test the product freely.
 - ▶ **Experienced users** and **long-time product support personnel** generate test cases based on practical insight.
 - ▶ **Specification-Based Testing (Black-Box Testing)**
 - ▶ Based entirely on **specifications** without examining the code.
 - ▶ Focuses on **input/output behavior** of the system.
 - ▶ Common techniques include: **Equivalence-Class Partitioning (ECP)**, **Boundary-Value Analysis (BVA)**.
 - ▶ **Code-Based Testing (White-Box Testing)**
 - ▶ Uses **knowledge of the source code** to design tests.
 - ▶ Aims to achieve **coverage goals** (e.g., statement, branch, or path coverage).
 - ▶ **Path analysis** is a common white-box testing technique.

Testing Techniques

▶ Existing Test Cases (Regression Testing)

- ▶ Reuses **previously created test cases** for earlier system versions.
- ▶ Confirms that **new changes** do not break existing functionality.

▶ Fault-Based Techniques

- ▶ **Error Guessing** - designing tests to trigger **likely faults** based on historical experience.
- ▶ **Error-Prone Analysis** - identifying **areas prone to defects** through **reviews and inspections**, then focusing testing efforts on those areas.

Testing and debugging

▶ Testing and Debugging: Key Difference

- ▶ **Debugging** is performed by **developers** to **identify the cause of defects** in code and **correct them**.
- ▶ **Testing** is a **systematic exploration** of a component or system with the primary goal of **finding and reporting defects**.

▶ Relationship Between Testing and Debugging

- ▶ Developers usually conduct **initial testing and debugging** to ensure the software is **stable enough** for more formal, rigorous testing.
- ▶ **Debugging** raises the **quality level** of the code to a point where testing can begin.
- ▶ **Testing** rigorously **examines system behaviour**, reports all detected defects, and ensures that **corrections are re-tested** to confirm effectiveness.

▶ Complementary Roles

- ▶ **Debugging** alone cannot guarantee that requirements are fully met.
- ▶ **Testing** provides **confidence** that the system behaves as specified.
- ▶ Both are essential:
 - ▶ **Debugging** → fixes the problems.
 - ▶ **Testing** → ensures the fixes and the system as a whole are correct.

General Testing Principles

- ▶ **Testing shows the presence, not absence, of defects**
- ▶ **Exhaustive testing is impossible**
 - ▶ Unless the application has an extremely simple logical structure and limited input, it is not possible to test all possible combinations of data input and circumstances.
 - ▶ Risk and priorities are used to concentrate on the most important aspects to test.
- ▶ **Early testing saves time and money**
 - ▶ As the deployment date approaches, time pressure increases dramatically.
 - ▶ In early testing, we are trying to find errors and defects before they are passed to the next stage of the development process.
 - ▶ If a problem is not corrected at the stage in which it is introduced, this leads to 'errors of migration'.
 - ▶ The result is rework. We need to rework not just the part where the mistake was made, but each subsequent part where the error has been replicated.

General Testing Principles

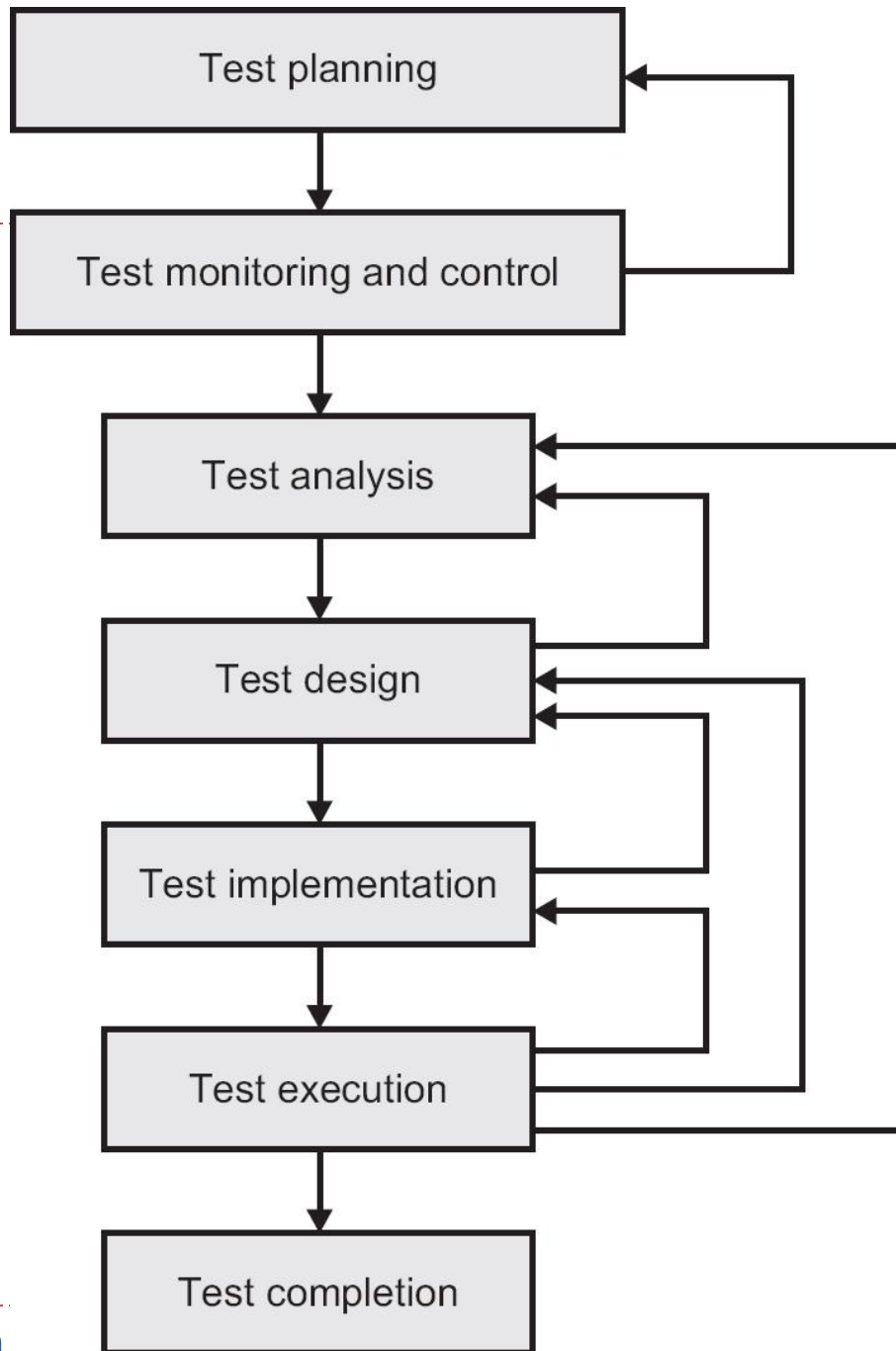
▶ Defects cluster together

- ▶ The spread of defects is typically not uniform. In a large application, it is often a small number of modules that exhibit the majority of the problems.
- ▶ This can be for a variety of reasons, some of which are:
 - ▶ system complexity;
 - ▶ the effects of change on change;
 - ▶ development staff experience;
 - ▶ development staff inexperience.
- ▶ This is the application of the Pareto principle to software testing: approximately 80 per cent of the problems are found in about 20 per cent of the modules.

▶ Be aware of the pesticide paradox

- ▶ Running the same set of tests continually will not continue to find new defects.
 - ▶ Developers will soon know that the test team always tests the boundaries of conditions, for example, so they will learn to test these conditions themselves before the software is delivered.
- ▶ Using other techniques will find different defects.

Test process



Test process

▶ Test Planning

- ▶ **Planning** defines *what* will be tested and *how* it will be achieved.
- ▶ It includes:
 - ▶ Establishing **test completion (exit) criteria**.
 - ▶ Defining the **objectives** of testing.
 - ▶ Selecting the **approach** to meet objectives, considering contextual limitations.
- ▶ A **test plan** is created, and **schedules** are drawn up to meet deadlines.
- ▶ The goal is to ensure testing is **organized, traceable, and aligned** with project objectives.

▶ Test Monitoring and Control

- ▶ **Monitoring** involves tracking progress — checking whether **achieved results** match **planned activities**.
- ▶ **Control** means taking **corrective actions** when deviations occur to meet objectives.
- ▶ Evaluating **exit criteria** during test execution may include:
 - ▶ Checking if the **required test coverage** has been achieved.
 - ▶ Assessing **system quality** based on test results and logs.
 - ▶ Deciding whether **additional tests** are necessary.

Test process

▶ Test Analysis

- ▶ **Purpose:** to examine the **test basis** to **identify what needs to be tested**.
- ▶ The outcome is a set of **test conditions**: the features, functions, or attributes to verify.
- ▶ **Examine the test basis**, which may include:
 - ▶ **Requirements documents** (functional and non-functional).
 - ▶ **Design or implementation details** describing the structure of the component or system.
 - ▶ **Implementation artifacts** - code, database metadata, queries, and configuration data.
 - ▶ **Risk analysis reports**, covering functional, non-functional, and structural aspects of the software.
- ▶ **Check the test basis for defects**, such as:
 - ▶ Ambiguities or missing information.
 - ▶ Inconsistencies, inaccuracies, or contradictions.
 - ▶ Unnecessary or redundant statements.
- ▶ **Identify and prioritize features or feature sets** to be tested.
- ▶ **Define and prioritize test conditions** for each identified feature.
- ▶ **Establish bi-directional traceability** between the **test basis** and **test conditions**, ensuring every requirement is covered and traceable back to its source.

Test process

▶ Test Design

- ▶ Test Analysis answers “*What to test?*”
Test Design answers “*How to test?*”
- ▶ Test conditions identified during analysis are now used to create test cases.
- ▶ A test case defines both the actions to perform and the expected result.

▶ Main Activities

- ▶ Use test conditions to create test cases, and prioritize them by importance or risk.
- ▶ Identify required test data to execute the test cases effectively.
- ▶ Design or configure the test environment, including necessary infrastructure, tools, and resources.
- ▶ Maintain bi-directional traceability between:
 - ▶ Test basis ↔ Test conditions
 - ▶ Test conditions ↔ Test cases
 - ▶ Test cases ↔ Test procedures
This ensures complete coverage and consistency throughout the testing process.

Test process

▶ Test Implementation

- ▶ Question: *“Do we have everything ready to run the tests?”*
- ▶ This phase acts as the **bridge between Test Design and Test Execution**.

▶ Main Activities

- ▶ **Create and prioritize test procedures**, and develop **automated test scripts** when applicable.
- ▶ **Assemble test suites** from test procedures and automated scripts.
- ▶ **Order test suites** in an **execution schedule** to ensure efficient use of time and resources.
- ▶ **Build and configure the test environment**, including:
 - ▶ **Test harnesses, simulators, stub interfaces, or service virtualization.**
 - ▶ **Any other necessary infrastructure components.**
- ▶ **Prepare and validate test data**, ensuring it is correctly loaded into the environment.
- ▶ **Verify and update bi-directional traceability** between:
 - ▶ **Test basis, test conditions, test cases, test procedures, and now test suites.**

Test process

▶ Test Execution

- ▶ **Purpose:** to carry out the **execution of tests** as planned and designed, recording results and identifying any defects that emerge.

▶ Main Activities

- ▶ **Record identification and version** of the item under test, this ensures traceability when defects are reported.
- ▶ **Execute tests**, either **manually** or using **automated test execution tools**.
- ▶ **Compare actual results** with the **expected results**.
- ▶ **Investigate any discrepancies** between expected and actual results to determine the cause:
 - ▶ It may indicate a **defect** in the software,
 - ▶ Or an **error** in the test itself (incorrect expected result or faulty test data).
- ▶ **Report defects** discovered during testing, documenting them clearly and reproducibly.
- ▶ **Record the result** of each test: *pass*, *fail*, or *blocked*.
- ▶ **Repeat tests** after corrections, data adjustments, or as part of **regression testing**.
- ▶ **Verify and update bi-directional traceability** among:
 - ▶ Test basis, test conditions, test cases, test procedures, and expected results.

Test process

▶ Test Completion

- ▶ Marks the **end of testing activities** for a given project or release.
- ▶ The focus is on **collecting results, preserving knowledge**, and ensuring that valuable test assets can be reused.
- ▶ Test completion may occur **at system release** or at **other project milestones**.
- ▶ The key objective is to ensure that **no critical information is lost**.

▶ Main Activities

- ▶ **Verify closure of defect reports** - ensure all necessary issues have been resolved or documented.
- ▶ **Create a Test Summary Report** to communicate results, coverage, and outstanding risks to stakeholders.
- ▶ **Archive the test environment, test data, and testware** (scripts, tools, documentation) for possible future reuse.
- ▶ **Analyze lessons learned** from testing activities to identify improvement areas.
- ▶ **Use collected data to enhance the testing process** and support continuous **test process maturity**.

Static testing

- ▶ Static testing tests software and artifacts without executing the code.
 - ▶ This includes requirements or specification documents (review), and the testing of code (static analysis).
 - ▶ Review is used to find and remove errors and ambiguities in documents before they are used in the development process, thus reducing defects in the code.
 - Giving a draft document to a colleague to read is the simplest example of a review.
 - ▶ Static analysis enables code to be analyzed for structural defects or systematic programming weaknesses that may lead to defects.
- ▶ Static techniques find the causes of failures rather than the failure itself, which is found during test execution.
 - ▶ Reviews are normally completed manually;
 - ▶ Static analysis is normally completed automatically using tools.

Artifact examined by Static Testing

- ▶ Any artifact can be examined using static testing techniques:
 - ▶ specifications such as business requirements, functional requirements, security requirements and non-functional requirements;
 - ▶ user stories and acceptance criteria;
 - ▶ architecture and design specifications;
 - ▶ code;
 - ▶ user guides;
 - ▶ web pages;
 - ▶ contracts, plans, schedules and budgets;
 - ▶ design models.

Benefits

- ▶ Static testing benefits may include:
 - ▶ detecting and correcting defects more efficiently, and prior to test execution;
 - ▶ identification of defects not easily found by dynamic testing;
 - ▶ preventing defects in design or coding by uncovering inconsistencies, ambiguities, contradictions, omissions, inaccuracies and redundancies in requirements;
 - ▶ increasing development productivity;
 - ▶ reducing development cost and time;
 - ▶ reducing dynamic testing cost and time;
 - ▶ reducing total cost of quality over the software's lifetime, due to fewer failures later in the life cycle or after delivery into live operation;
 - ▶ improving communication between team members in the course of participating in reviews.

Review Process

▶ Informal Reviews

- ▶ Do not follow a defined process and have no formal documentation as output.
- ▶ Usually consist of **ad-hoc discussions** or quick evaluations (e.g., peer reviews, pair programming).
- ▶ Focus on **early feedback** and **knowledge sharing** rather than compliance.

▶ Formal Reviews

- ▶ Characterized by **structured procedures**, **documented results**, and **defined roles**.
- ▶ Conducted by a **team** rather than an individual.
- ▶ Often used in **regulated environments** or **mature development processes** where traceability and accountability are essential.

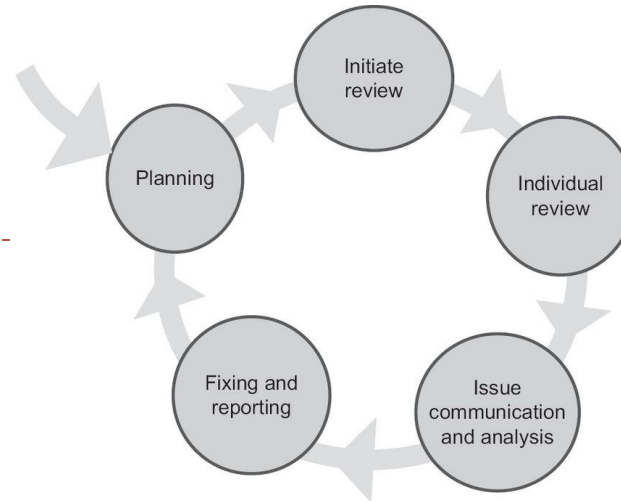
▶ Determining the Level of Formality

- ▶ The **Software Development Life Cycle (SDLC)** model in use.
- ▶ The **maturity** of the development process (more mature → more formal).
- ▶ The **complexity** of the work product being reviewed.
- ▶ Any **legal or regulatory requirements**.

Review Process

- ▶ **Typical Review Objectives**
 - ▶ **Finding defects**
 - ▶ **Gaining understanding** of the product or process
 - ▶ **Generating discussion** among team members
 - ▶ **Education** and knowledge transfer
 - ▶ **Decision-making** by consensus
- ▶ The way a review is conducted depends its specific objective:
 - ▶ a review aimed at finding defects is different from one that is aimed at gaining understanding of a document.
- ▶ All reviews exhibit the same basic elements of process:
 - ▶ The document under review is studied by the reviewers.
 - ▶ Reviewers identify issues and inform the author either verbally or in a documented form, which might be as formal as raising a defect report or as informal as annotating the document under review.
 - ▶ The author decides on any action to take in response to the comments, and updates the document accordingly.
- ▶ This basic process is always present, but in the more formal reviews it is elaborated to include additional stages and more attention to documentation and measurement.

Product Review Process



► Review Planning

- **Purpose:** to define *what* will be reviewed, *who* will participate, and *how* the review will be conducted.
- Planning ensures that the review process is **organized, efficient, and goal-oriented**.

► Main Planning Activities

- **Define the scope:** State the **purpose** of the review. Identify **which documents or sections** will be reviewed. Specify the **quality characteristics** to be evaluated (e.g., clarity, consistency, completeness).
- **Estimate effort and timeframe.**
- **Identify review characteristics:** Define the **type of review, roles, activities**, and any **checklists** to be used.
- **Select participants:** the **reviewers** based on expertise and relevance and a **moderator (review leader)** early in the process.
- **Assign roles:** Each reviewer is given a **specific focus or area of responsibility** to ensure thorough coverage.
- **Define entry and exit criteria.**

Product Review Process

▶ Initiate Review

- ▶ **Distribute artifacts:** share all relevant documents or materials with reviewers.
- ▶ **Explain the review scope and objectives** to participants.
- ▶ Clarify the **process, roles, and expected work products**.
- ▶ **Answer questions** raised by the review team to ensure common understanding.

▶ Individual Review

- ▶ Each reviewer examines **all parts of the work product**, including any **source documents**.
- ▶ Reviewers **identify potential defects**, raise **questions**, and record **comments**.
- ▶ Individual preparation ensures that review meetings are **focused and efficient**.

▶ Issue Communication and Analysis

- ▶ **Communicate findings** either during a **review meeting** or through a **defect log**.
- ▶ **Analyze potential defects:**
 - ▶ If confirmed, assign **ownership for correction**.
 - ▶ Evaluate whether **quality criteria** have been met.
- ▶ **Assess review results** against **exit criteria** to decide whether the product:
 - ▶ Needs **rejection and rework**, or
 - ▶ Can be **accepted with minor updates**.

Product Review Process

▶ Fixing and Reporting

▶ Fixing Defects

- ▶ The **author** corrects all **defects that were found and agreed upon** during the review.
- ▶ **Updated defect status** is recorded — this step is **mandatory** in **formal reviews** and **optional** in informal ones.

▶ Gathering Metrics

- ▶ Collect data such as:
 - **Time spent** on the review.
 - **Number of defects found**.
- ▶ Metrics help demonstrate that:
 - The **quality of the reviewed item** has improved.
 - The **review added value** to later stages of development.
 - For example, how much **cost and effort** were saved by detecting a defect early, before reaching **user acceptance testing** or production.

▶ Checking Exit Criteria and Closure

- ▶ The **moderator** verifies that all **exit criteria** have been met.
- ▶ When satisfied, the **review is officially closed**.
- ▶ The **work product** is formally **accepted** once all required corrections are completed.

Roles and Responsibilities

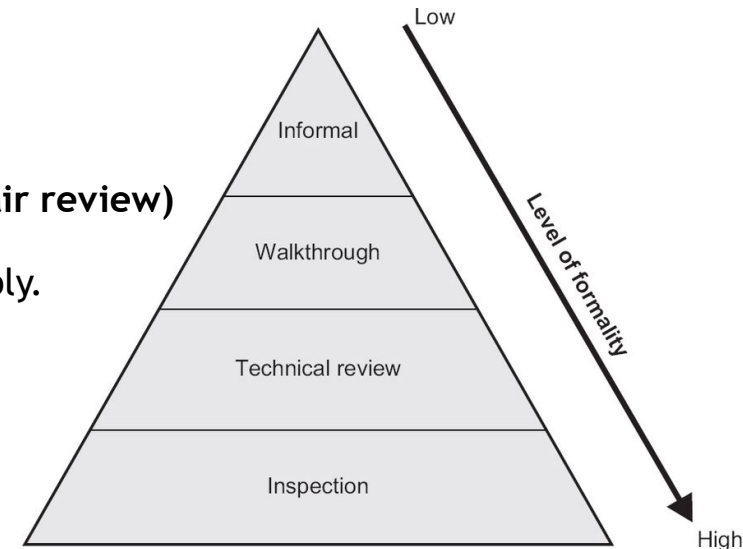
- ▶ **General Reviewer Role**
- ▶ Each reviewer examines the document from a **specific assigned perspective**.
- ▶ Reviewers may use **checklists** to guide their focus:
 - ▶ *Perspective-based checklists* (e.g., user, maintainer, tester, operations).
 - ▶ *General checklists* (e.g., common requirements or design problems).
- ▶ **Specific Roles in the Review Process**
 - ▶ **Author**
 - ▶ Writes or is responsible for the document under review.
 - ▶ Fixes **agreed defects** identified during the review.
 - ▶ **Manager**
 - ▶ Decides **what will be reviewed** and **assigns personnel**.
 - ▶ Ensures **sufficient time** is allocated in the project plan.
 - ▶ Monitors the **cost-effectiveness** of the review.
 - ▶ Confirms **objectives are met** and takes corrective action if not.
 - ▶ Normally does **not participate** in the review unless their expertise adds value.

Roles and Responsibilities

- ▶ **Facilitator (Moderator)**
 - ▶ Ensures the **review process runs efficiently**.
 - ▶ **Mediates discussions** and resolves conflicts.
 - ▶ Makes the **final decision** on whether to release the updated document.
- ▶ **Review Leader**
 - ▶ Responsible for **planning, conducting, and following up** on the review meeting.
 - ▶ Coordinates participants and ensures that objectives and deadlines are met.
- ▶ **Reviewers**
 - ▶ Subject Matter Experts (SMEs), checkers, or inspectors.
 - ▶ Review the product from **different technical or business perspectives**.
 - ▶ Identify and describe **findings or defects** during preparation and meetings.
- ▶ **Scribe (Recorder)**
 - ▶ Attends the review meeting.
 - ▶ **Documents all issues, defects, and open points** discussed.
 - ▶ Ensures accurate recording for traceability and follow-up actions.

Types of Review

- ▶ **Informal Review (Least Formal - Buddy check, Pairing, Pair review)**
- ▶ **Main purpose:** to detect potential defects early and cheaply.
- ▶ **Additional purposes:**
 - ▶ Generate new ideas or alternative solutions.
 - ▶ Quickly resolve issues or ambiguities.
- ▶ **Key Characteristics**
 - ▶ No formal process is required; reviews are ad hoc and flexible.
 - ▶ May or may not include a meeting between participants.
 - ▶ Can be performed by a **colleague of the author** or shared with a **wider audience**.
 - ▶ Documentation of the review is optional.
 - ▶ Use of a checklist is optional and depends on the context.
 - ▶ Effectiveness may vary depending on the reviewer's **technical skill** and familiarity with the topic.



Types of Review

- ▶ **Walkthrough**
- ▶ **Main Purpose:**
 - ▶ To find defects, improve the software product, and evaluate conformance to standards and specifications.
 - ▶ To consider alternative implementations or design options.
- ▶ **Additional goals include:**
 - ▶ **Sharing ideas** about techniques or coding styles.
 - ▶ **Training participants** through discussion and peer learning.
 - ▶ **Building consensus** on the product's quality or direction.
- ▶ **Key Characteristics**
 - ▶ The **author leads** the meeting and presents the work product to peers.
 - ▶ The **scribe** (not the author) is **mandatory** to document findings and decisions.
 - ▶ **Preparation** by reviewers, **use of a checklist**, and **production of a review report** are all **optional**.
 - ▶ Attendees are typically **members of the author's peer group**.
 - ▶ **Sessions are open-ended** and can vary from **informal to quite formal** in practice.
 - ▶ Walkthroughs often include **scenario-based discussions** or **dry runs** of code or processes.
 - ▶ May produce **defect logs** and **review reports** summarizing findings.

Types of Review

- ▶ **Technical Review**
- ▶ **Main Purpose:** To **gain consensus** among peers and **detect potential defects** in the work product.
- ▶ **Additional Purposes**
 - ▶ Evaluate **quality** and **build confidence** in the reviewed item.
 - ▶ Generate **new ideas** and promote technical improvement.
 - ▶ **Motivate authors** to enhance future work products.
 - ▶ Encourage consideration of **alternative designs or implementations**.
- ▶ **Key Characteristics**
 - ▶ The review process is **documented** and follows a **defined defect detection procedure**.
 - ▶ Involves **peers and technical experts** familiar with the subject matter.
 - ▶ **Individual preparation** by reviewers **before the meeting** is required.
 - ▶ A **review meeting** is *optional*; if held, it is ideally **facilitated by a trained moderator**.
 - ▶ A **scribe** is **mandatory** (preferably not the author).
 - ▶ **Checklists** may be used but are **optional**.
 - ▶ **Defect logs** and **review reports** are typically **produced** to capture findings and results.

Types of Review

- ▶ **Inspection (Most Formal Review Type)**
- ▶ **Main Purpose**
 - ▶ To detect potential defects, evaluate quality, and build confidence in the work product.
 - ▶ To prevent future defects through author learning and root cause analysis.
- ▶ **Key Characteristics**
 - ▶ The process is highly formal, based on defined rules, checklists, and entry/exit criteria.
 - ▶ Pre-meeting preparation is essential — reviewers must study source documents for consistency.
 - ▶ Reviewers are peers of the author or subject-matter experts from related disciplines.
 - ▶ Specific entry and exit criteria control when the inspection starts and when it can be closed.
 - ▶ A scribe is mandatory to record all findings.
 - ▶ Meetings are led by a trained facilitator (not the author).
 - ▶ The author cannot act as leader or scribe.
 - ▶ Reviewers examine the product individually and collectively according to defined roles.
 - ▶ All potential defects are logged, and a formal review report is produced.
 - ▶ Metrics are collected to improve both the software development and the inspection process itself.

Applying Review Techniques

- ▶ **Ad Hoc**
 - ▶ Common in **less formal reviews**: provides **little or no guidance** on what to look for or how to perform the review.
 - ▶ Depends heavily on the **reviewer's skill and experience**.
 - ▶ Can lead to **duplication** of defects found or **missed issues** due to lack of structure.
- ▶ **Checklist-Based**
 - ▶ Uses a **structured checklist** to ensure a **systematic review**.
 - ▶ The checklist defines: **Activities** and **Types of defects** to look for (e.g., typos, inconsistencies, technical errors).
 - ▶ Improves **consistency and completeness** of the review process.
- ▶ **Scenarios and Dry Runs**
 - ▶ Reviewers simulate **expected product usage** through **scenarios**.
 - ▶ Provides **clearer guidance** for identifying specific defects or simulating real user behavior.
- ▶ **Role-Based**
 - ▶ The artifact is reviewed from the **perspective of a defined role** (e.g., user administrator, system administrator, tester).
 - ▶ Helps ensure that **role-specific concerns** are identified and addressed.
- ▶ **Perspective-Based**
 - ▶ The reviewer adopts the **viewpoint of different stakeholders** (e.g., end user, designer, marketer).
 - ▶ Encourages a **broader and more balanced evaluation** of the product.



Applying Review Techniques

▶ Success Factors for Reviews

- ▶ Organizational Factors
- ▶ People-Related Factors

▶ Examples of Organizational Success Factors

- ▶ Each review should have a **clearly defined and agreed objective**.
- ▶ **Review techniques** (formal or informal) must be **appropriate** for:
 - ▶ The **type and level** of software artifacts, and
 - ▶ The **skills and experience** of reviewers.
- ▶ Techniques such as **checklist-based** and **role-based** reviews are particularly effective for **defect identification**.
- ▶ **Checklists** should be **focused on main risk areas** and **kept up to date**.
- ▶ **Review participants** must have **sufficient time** to prepare properly.
- ▶ **Reviews should be scheduled** in advance, with **adequate notice** to all participants.
- ▶ **Management support** is critical – for example:
 - ▶ Allocating **time and resources** for review activities in the project plan.
 - ▶ Encouraging a **quality-oriented culture** within the team.

Applying Review Techniques

- ▶ **People-Related Success Factors for Reviews**
- ▶ The **right people** are involved to ensure that **review objectives are achieved**: ideally, individuals with **diverse skill sets** and perspectives, including representatives of the **user community**.
- ▶ **Testers participate** as reviewers to:
 - ▶ Learn about the product,
 - ▶ Design more effective tests, and
 - ▶ Develop those tests **earlier in the process**.
- ▶ **Adequate time** is allocated for **each participant** to prepare and contribute effectively.
- ▶ **Defects are appreciated** as opportunities for **improvement**, not criticism.
- ▶ **Review meetings are well managed** and perceived as a **valuable use of time**.
- ▶ A **climate of trust** is maintained:
 - ▶ The review evaluates **the work product**, not **the person**.
 - ▶ **Negative body language** and personal attacks are avoided.
- ▶ **Adequate training** is provided, particularly for **formal review techniques**.
- ▶ A **review culture** is encouraged – one that emphasizes **learning, collaboration, and continuous process improvement**.

Categories of Test Case Design Techniques

- ▶ There are very many ways to design test cases that can be grouped into three categories:
 - ▶ **Black-box techniques:** deriving test cases directly from a specification or a model of a system or proposed system.
 - ▶ based on an analysis of the test basis documentation, including functional and non-functional aspects.
 - ▶ do not use any information regarding the internal structure of the component or system under test.
 - ▶ **White-box techniques:** deriving test cases directly from the structure of a component or system.
 - ▶ It concentrates on tests based on the code written to implement a component or system.
 - ▶ **Experience-based techniques:** deriving test cases from the tester's experience of similar systems and general experience of testing.

Black-Box Testing /Behavioral Testing

- ▶ The technique of testing without having any knowledge of the interior workings of the application is called black-box testing.
- ▶ Typically, while performing a black-box test, a tester will interact with the system's user interface by providing inputs and examining outputs without knowing how and where the inputs are worked upon.

Advantages	Disadvantages
Well suited and efficient for large code segments.	Limited coverage, since only a selected number of test scenarios is actually performed.
Code access is not required.	Inefficient testing, due to the fact that the tester only has limited knowledge about an application.
Clearly separates user's perspective from the developer's perspective through visibly defined roles.	Blind coverage, since the tester cannot target specific code segments or error-prone areas.
Large numbers of moderately skilled testers can test the application with no knowledge of implementation, programming language, or operating systems.	The test cases are difficult to design.

White-Box Testing

- ▶ White-box testing is the detailed investigation of internal logic and structure of the code.
 - ▶ White-box testing is also called **glass testing** or **open-box testing**.
 - ▶ In order to perform **white-box** testing on an application, a tester needs to know the internal workings of the code.
- ▶ The tester needs to have a look inside the source code and find out which unit/chunk of the code is behaving inappropriately.

Advantages	Disadvantages
As the tester has knowledge of the source code, it becomes very easy to find out which type of data can help in testing the application effectively.	Due to the fact that a skilled tester is needed to perform white-box testing, the costs are increased.
It helps in optimizing the code.	Sometimes it is impossible to look into every nook and corner to find out hidden errors that may create problems, as many paths will go untested.
Extra lines of code can be removed which can bring in hidden defects.	It is difficult to maintain white-box testing, as it requires specialized tools like code analyzers and debugging tools.
Due to the tester's knowledge about the code, maximum coverage is attained during test scenario writing	

Black-Box Testing

- ▶ We need techniques that can explore the specified behaviour systematically and thoroughly in a way that is as efficient as we can make it.
 - ▶ equivalence partitioning;
 - ▶ boundary value analysis;
 - ▶ decision table testing;
 - ▶ use case testing.

Equivalence Class Partitioning

- ▶ Equivalence class partitioning is a specification-based (i.e., black-box technique).
 - ▶ It is based on dividing the input into several classes that are equivalent for the purposes of finding errors.
 - ▶ If a program fails for one member of the class, then we expect it to fail for all other members;
 - ▶ If it is correct for one member of the class, we expect it to be correct for all members.
- ▶ The equivalence classes are determined by looking at the requirements specification and by the tester's intuition, without looking at the implementation.
 - ▶ The equivalence classes will cover the complete input domain.
 - ▶ At no point does any class overlap with any other class

Equivalence Class Partitioning (2)

Example: pick “larger” of two integers and -----

Class	Representative
First > Second	10,7
Second > First	8,12
First = second	36, 36

- ▶ Equivalence class partitioning is most useful when the requirements are expressed as a series of conditions.
- ▶ It is less useful when the requirements seem to call for loops.
 - ▶ For example, if we are testing a function that sums all the elements in an array or vector, it would be hard to decide what the equivalence classes are.

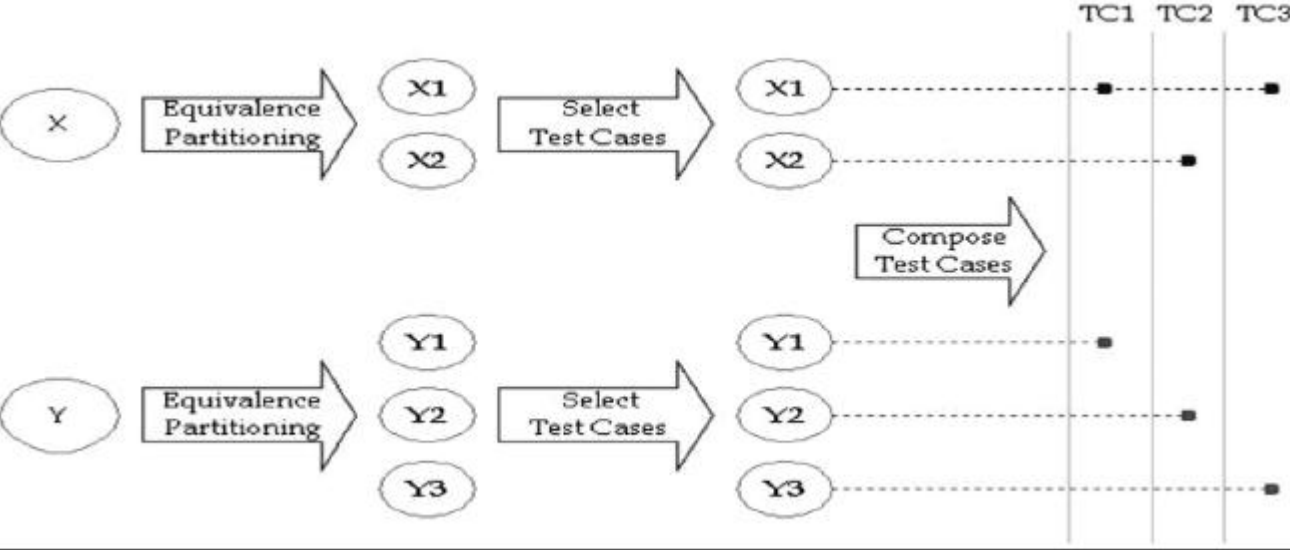
Equivalence Class Partitioning (3)

- ▶ There are
 - ▶ *valid equivalence classes*: describe situations that the system should handle normally.
 - ▶ *invalid equivalence classes*: describe invalid situations that the system should reject.
- ▶ Usually, we are working with more sets of equivalence classes at one time.
 - ▶ We create one set of valid tests with one valid member from each equivalence partition.
 - ▶ We continue this process until each valid class is represented in at least one valid test.
- ▶ Next, we can create invalid tests.
 - ▶ We select one invalid member for one equivalence partition and a valid member for every other partition.
 - ▶ Don't combine multiple invalid members in a single test: one invalid value might mask the incorrect handling of another invalid value.
 - ▶ We continue the process until each invalid class is represented in at least one invalid test.
- ▶ Every class, both valid and invalid, is represented in at least one test.

Avoiding Equivalence Partitioning Errors

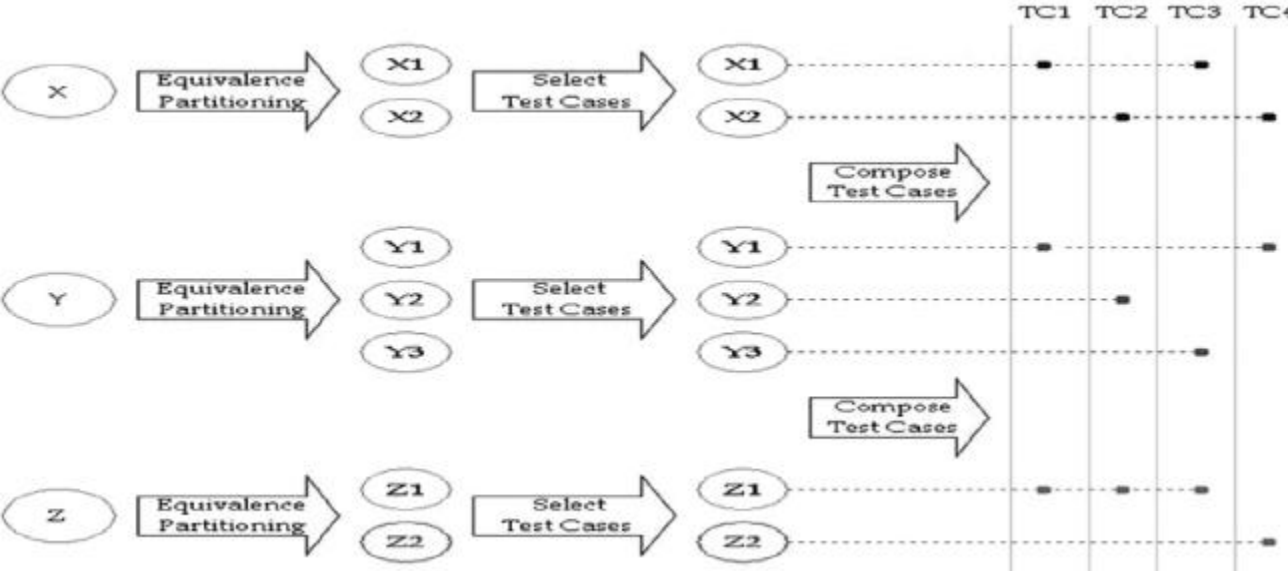
- ▶ The equivalence partitions have to be disjoint.
 - ▶ That is, no two of the subsets can have one or more members in common.
- ▶ None of the equivalence partitions may be empty.
 - ▶ That is, if the equivalence partitioning operation produces a subset with no members, that's hardly very useful for testing.
- ▶ The union of the subsets produced by equivalence partitioning must be the same as the original set that was equivalence partitioned.
 - ▶ In other words, equivalence partitioning does not generate “spare” subsets that are somehow disposed of in the process.

Composing Tests



- ▶ We partition set X into two subsets, X1 and X2.
- ▶ We partition set Y into three subsets, Y1, Y2, and Y3.
- ▶ We select test values from each of the five subsets, X1, X2, Y1, Y2, and Y3.
- ▶ We then compose three tests, since we can combine the values from the X subsets with values from the Y subsets (assuming the values are independent and all valid).
- ▶ Example: imagine you are testing a browser-based application.
 - ▶ You are interested in two browsers, Chrome and Firefox.
 - ▶ You are interested in three connection types, Wi-Fi, DSL, and cable modem.
 - ▶ Since the browser and the connection types are independent, we can create three tests.
 - ▶ In each of the tests, one of the connection types and one of the browser types will be represented. One of the browser types will be represented twice.

Composing Tests (2)



- ▶ In some cases, values are not valid.
 - ▶ We are testing a project management application, something like Microsoft Project.
 - ▶ X is the type of event we're dealing with, task (X1) or a milestone (X2).
 - ▶ Y is the start date of the event, which can be in the past (Y1), today (Y2), or in the future (Y3).
 - ▶ Z is the end date of the event, which can be either on or after the start date (Z1) or before the start date (Z2). Of course, Z2 is invalid.

Example: HELLOCARMS system

The screenshot shows a web form for the HELLOCARMS system. It contains three main sections:

- Apply for Product?**: A dropdown menu with the placeholder text "{Select a product}" and a downward arrow. The dropdown is open, showing three options: "Home equity loan", "Home equity line of credit", and "Reverse mortgage".
- Existing Checking?**: A dropdown menu with the placeholder text "{Select Yes or No}" and a downward arrow. The dropdown is open, showing two options: "Yes" and "No". To the right of this dropdown is a text input field with the placeholder text "{If Yes input account number}".
- Existing Savings?**: A dropdown menu with the placeholder text "{Select Yes or No}" and a downward arrow. The dropdown is open, showing two options: "Yes" and "No". To the right of this dropdown is a text input field with the placeholder text "{If Yes input account number}".

1. Show the equivalence partitions for each of the three pieces of information, indicating valid and invalid members
2. Create test cases to cover these partitions, keeping in mind the rules about combinations of valid and invalid members

HELLOCARMS system

- ▶ This screen asks for three pieces of information:
 - ▶ 1. The product being applied for, which is one of the following:
 - ▶ a. Home equity loan
 - ▶ b. Home equity line of credit
 - ▶ c. Reverse mortgage
 - ▶ 2. Whether someone has an existing Globobank checking account, which is either Yes or No.
 - ▶ 3. Whether someone has an existing Globobank savings account, which is either Yes or No.
- ▶ If the user indicates an existing Globobank account, then the user must enter the corresponding account number. This number is validated against the bank's central database upon entry. If the user indicates no such account, the user must leave the corresponding account number field blank.
- ▶ If the fields are valid, including the account number fields, then the screen will be accepted. If one or more fields are invalid, an error message is displayed

Example equivalence Partitioning

- ▶ For the application-product field, the equivalence partitions are:
 - ▶ 1 Home equity loan / 2 Home equity line of credit 3 Reverse mortgage
- ▶ The existing checking account information partitions are:
 - ▶ 1 Yes-Valid / 2 Yes-Invalid / 3 No-Blank / 4 No-Nonblank
- ▶ The existing savings account information partitions are:
 - ▶ 1 Yes-Valid / 2 Yes-Invalid / 3 No-Blank / 4 No-Nonblank

Example equivalence Partitioning (2)

Inputs	1	2	3	4	5	6	7
Product	HEL	LOC	RM	HEL	LOC	RM	HEL
Existing Checking?	Yes	No	No	Yes	No	No	No
Checking Account	Valid	Blank	Blank	Invalid	Nonblank	Blank	Blank
Existing Savings?	No	Yes	No	No	No	Yes	No
Savings Account	Blank	Valid	Blank	Blank	Blank	Invalid	Nonblank
Outputs							
Accept?	Yes	Yes	Yes	No	No	No	No
Error?	No	No	No	Yes	Yes	Yes	Yes

Product:

#	Partition	Test Case
1	Home equity loan (HEL)	1
2	Home equity line of credit (LOC)	2
3	Reverse mortgage (RM)	3

Checking:

#	Partition	Test Case
1	Yes-Valid	1
2	Yes-Invalid	4
3	No-Blank	2
4	No-Nonblank	5

Savings:

#	Partition	Test Case
1	Yes-Valid	2
2	Yes-Invalid	6
3	No-Blank	1
4	No-Nonblank	7

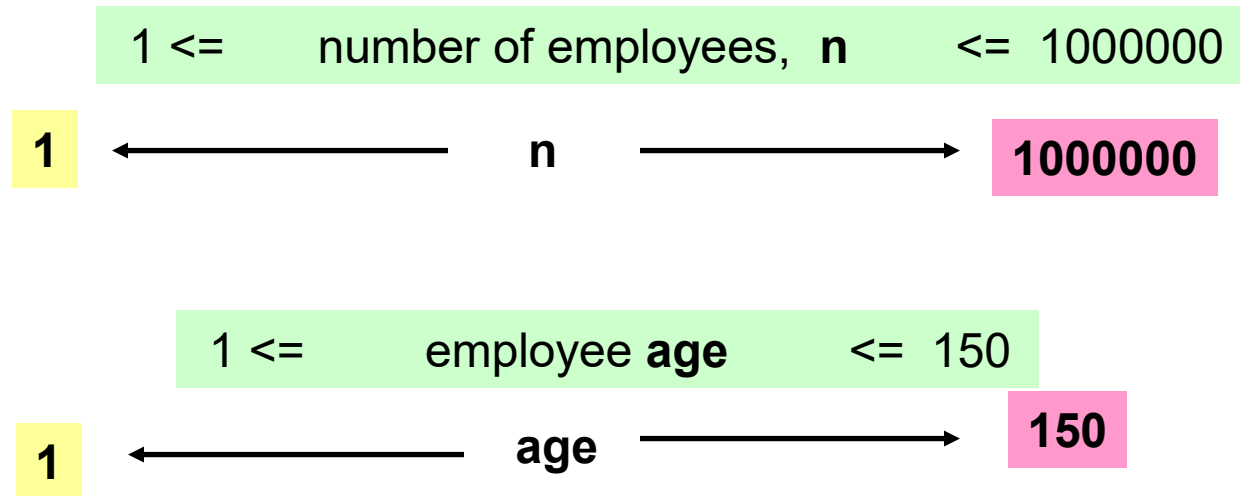
Boundary Value Analysis

- ▶ Many errors are actually made at the boundaries rather than under normal conditions.
 - ▶ Boundary value analysis uses the same classes as equivalence partitioning, testing at the boundaries rather than an element from the class.
 - ▶ A complete boundary value analysis calls for testing the boundary, the immediate case inside the boundary, and the immediate case outside of the boundary.
 - ▶ Because the equivalence classes usually form a progression of intervals, there will be overlap in the test cases. We usually consider the test case as coming from the interval to which it belongs.
- ▶ This technique may generate a high number of test cases. You can usually reduce test cases by just testing for above and below the boundaries.

Two Boundary Values or Three?

- ▶ Just two boundary values per class.
 - ▶ The boundary lies between the largest member of one equivalence class and the smallest member of the equivalence class above it.
 - ▶ The boundary itself doesn't correspond to any member of any class.
- ▶ However, some authors say there are three boundary values
 - ▶ You select the value itself as the middle boundary value, then you add the smallest possible amount to that value and subtract the smallest possible amount to that value to generate the two other boundary values.

Three Boundary Value Analysis - example



The “basic” boundary value testing for a value would include:

1. - at the “minimum” boundary
2. - immediately above minimum
3. - between minimum and maximum (nominal)
4. - immediately below maximum
5. - at the “maximum” boundary

**** note that we did not include the “outside” of the boundaries here****

Example: HELLOCARMS system

Loan amount?

Whole dollar amounts (no cents).
System will round to nearest \$100.

Property value?

Whole dollar amounts (no cents).
System will round to nearest \$100.

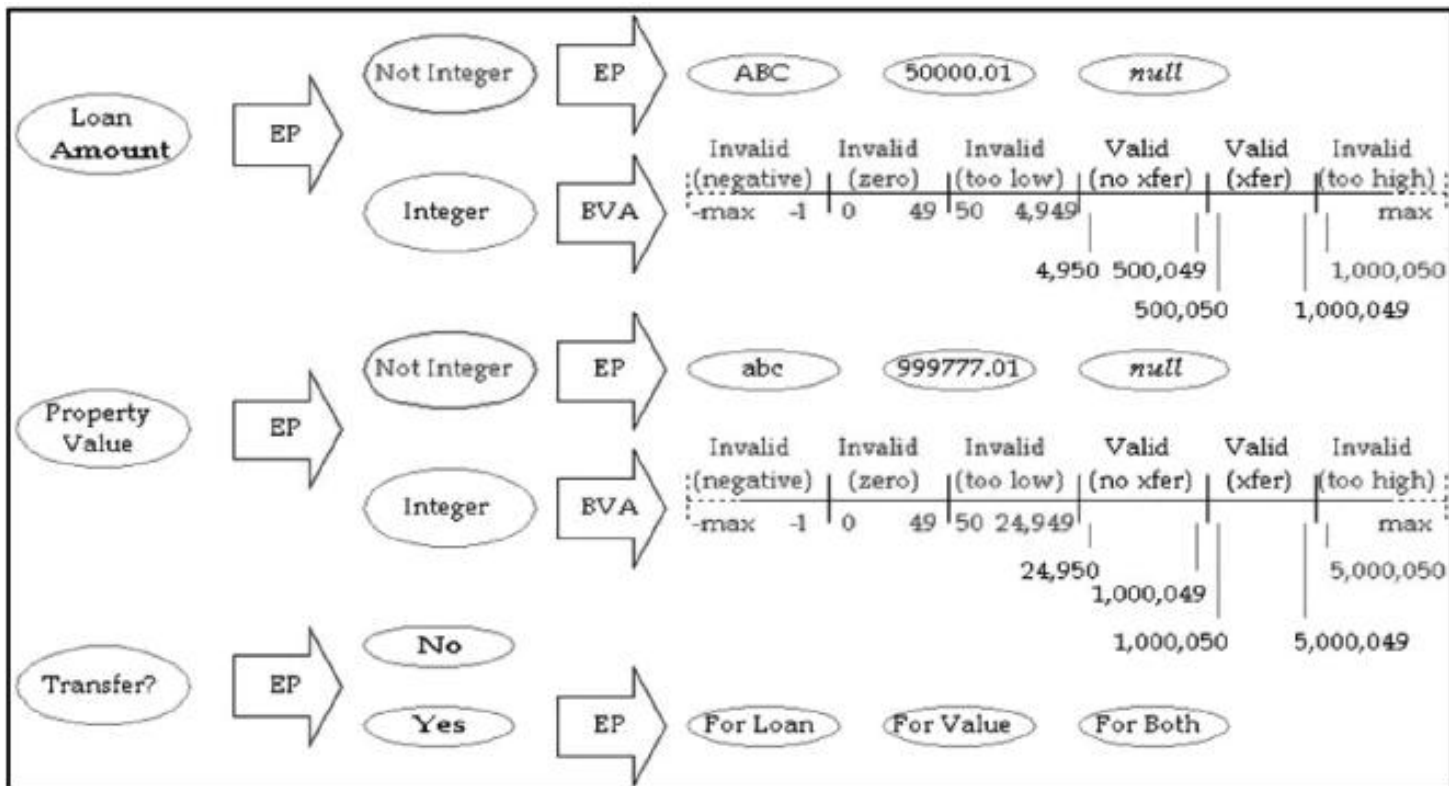
If the fields are valid, then the screen will be accepted.

If one or both fields are invalid, an error message is displayed.

- ▶ Loan amount
 - ▶ The minimum loan amount is \$5,000
 - ▶ The maximum loan amount is \$1,000,000
- ▶ Property value
 - ▶ The minimum property value is \$25,000
 - ▶ The maximum property value is \$5,000,000

Direct the Telephone Banker to transfer the call to a Senior Telephone Banker if an application has a loan amount greater than \$500,000 or a property with value greater than \$1,000,000; such loans require additional management approval.

Equivalence partitions and boundary values



Loan Amount

#	Partition	Boundary Value
1	Letter: ABC	-
2	Decimal: 50,000.01	-
3	Null	-
4	Invalid (negative)	-max
5	Invalid (negative)	-1
6	Invalid (zero)	0
7	Invalid (zero)	49
8	Invalid (too low)	50
9	Invalid (too low)	4,949
10	Valid (no transfer)	4,950
11	Valid (no transfer)	500,049
12	Valid (transfer)	500,050
13	Valid (transfer)	1,000,049
14	Invalid (too high)	1,000,050
15	Invalid (too high)	max

Property Value

#	Partition	Boundary Value
1	Letter: abc	-
2	Decimal: 999,777.01	-
3	Null	-
4	Invalid (negative)	-max
5	Invalid (negative)	-1
6	Invalid (zero)	0
7	Invalid (zero)	49
8	Invalid (too low)	50
9	Invalid (too low)	24,949
10	Valid (no transfer)	24,950
11	Valid (no transfer)	1,000,049
12	Valid (transfer)	1,000,050
13	Valid (transfer)	5,000,049
14	Invalid (too high)	5,000,050
15	Invalid (too high)	max

Transfer call

#	<i>Partition</i>
1	No
2	Yes, for the loan amount
3	Yes, for the property value
4	Yes, for both

Test cases

Inputs	1	2	3	4	5	6
Loan amount	4,950	500,050	500,049	1,000,049	ABC	50,000.01
Property value	24,950	1,000,049	1,000,050	5,000,049	100,000	200,000
Outputs						
Accept?	Y	Y	Y	Y	N	N
Transfer?	N	Y (loan)	Y (prop)	Y (both)	-	-

Inputs	7	8	9	10	11	12
Loan amount	null	100,000	200,000	300,000	-max	-1
Property value	300,000	abc	999,777.01	null	400,000	500,000
Outputs						
Accept?	N	N	N	N	N	N
Transfer?	-	-	-	-	-	-

Inputs	13	14	15	16	17	18
Loan amount	0	49	50	4,949	1,000,050	max
Property value	600,000	700,000	800,000	900,000	1,000,000	1,100,000
Outputs						
Accept?	N	N	N	N	N	N
Transfer?	-	-	-	-	-	-



Test cases (2)

Inputs	19	20	21	22	23	24
Loan amount	400,000	500,000	600,000	700,000	800,000	900,000
Property value	-max	-1	0	49	50	24,949
Outputs						
Accept?	N	N	N	N	N	N
Transfer?	-	-	-	-	-	-

Inputs	25	26	27	28	29	30
Loan amount	1,000,000	555,555				
Property value	5,000,050	max				
Outputs						
Accept?	N	N				
Transfer?	-	-				

Loan amount

#	Partition	Boundary Value	Test Case
1	Letter: ABC	-	5
2	Decimal: 50,000.01	-	6
3	Null	-	7
4	Invalid (negative)	-max	11
5	Invalid (negative)	-1	12
6	Invalid (zero)	0	13
7	Invalid (zero)	49	14
8	Invalid (too low)	50	15
9	Invalid (too low)	4,949	16
10	Valid (no transfer)	4,950	1
11	Valid (no transfer)	500,049	3
12	Valid (transfer)	500,050	2
13	Valid (transfer)	1,000,049	4
14	Invalid (too high)	1,000,050	17
15	Invalid (too high)	max	18

Property value

#	Partition	Boundary Value	Test Case
1	Letter: abc	-	8
2	Decimal: 999,777.01	-	9
3	Null	-	10
4	Invalid (negative)	-max	19
5	Invalid (negative)	-1	20
6	Invalid (zero)	0	21
7	Invalid (zero)	49	22
8	Invalid (too low)	50	23
9	Invalid (too low)	24,949	24
10	Valid (no transfer)	24,950	1
11	Valid (no transfer)	1,000,049	2
12	Valid (transfer)	1,000,050	3
13	Valid (transfer)	5,000,049	4
14	Invalid (too high)	5,000,050	25
15	Invalid (too high)	max	26

Transfer Decision

#	<i>Partition</i>	<i>Test Case</i>
1	No	1
2	Yes, for the loan amount	2
3	Yes, for the property value	3
4	Yes, for both	4

Decision Tables

- ▶ Decision tables allow us to test the way that various combinations of conditions interact to produce certain results happening and not happening.
 - ▶ The table arranges the combinations of conditions in columns, with the conditions on top and the associated actions taken and not taken shown at the bottom.
 - ▶ We have at least one test for each column
 - ▶ Boundary value analysis or equivalence partitioning on one or more conditions can create multiple tests
- ▶ Decision tables are not typically useful for unit testing (too simple).
- ▶ Conceptually, decision tables express the rules that govern handling of transactional situations.
- ▶ By their concise structure, they make it to design tests for those rules, usually at least one test per rule.

Example: Decision Table

- ▶ Validating payment information: credit card type, card number, card security code, expiration month, expiration year, and cardholder name.
- ▶ Once that information goes to the credit card processing company for validation, how can we test that?
 - ▶ Does the named person hold the credit card entered, and is the other information correct?
 - ▶ Is the account still active or has it been canceled?
 - ▶ Is the person within or over their limit?
 - ▶ Is the transaction coming from a normal or a suspicious location?

Decision Table

Conditions	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Real account?	Y	Y	Y	Y	Y	Y	Y	Y	N	N	N	N	N	N	N	N
Active account?	Y	Y	Y	Y	N	N	N	N	Y	Y	Y	Y	N	N	N	N
Within limit?	Y	Y	N	N	Y	Y	N	N	Y	Y	N	N	Y	Y	N	N
Location okay?	Y	N	Y	N	Y	N	Y	N	Y	N	Y	N	Y	N	Y	N
Actions																
Approve?	Y	N	N	N	N	N	N	N	N	N	N	N	N	N	N	N
Call cardholder?	N	Y	Y	Y	N	Y	Y	Y	N	N	N	N	N	N	N	N
Call vendor?	N	N	N	N	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y

- ▶ The decision table shows how the conditions interact to determine which of the following three actions will occur:
 - ▶ Should we approve the transaction?
 - ▶ Should we call the cardholder?
 - ▶ Should we call the vendor?

Not all the combinations have a meaning or are useful to be tested

Collapsing a Decision Table

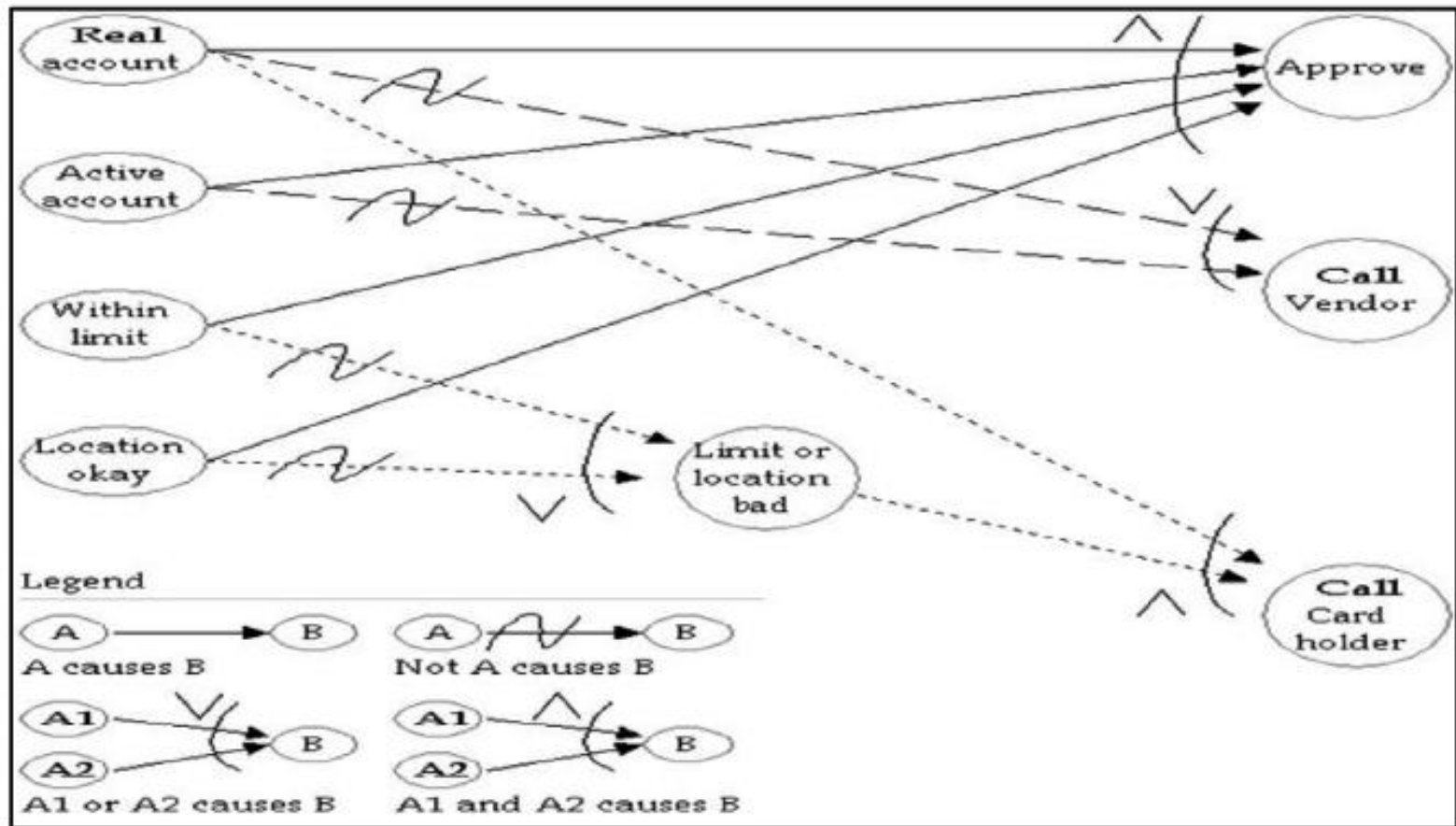
- ▶ We can collapse the decision table, combining columns, to achieve a more concise decision table.
- ▶ In any situation where the value of one or more particular conditions can't affect the actions for two or more combinations of conditions, we can collapse the decision table.

Conditions	1	2	3	5	6	7	9
Real account?	Y	Y	Y	Y	Y	Y	N
Active account?	Y	Y	Y	N	N	N	-
Within limit?	Y	Y	N	Y	Y	N	-
Location okay?	Y	N	-	Y	N	-	-
Actions							
Approve?	Y	N	N	N	N	N	N
Call cardholder?	N	Y	Y	N	Y	Y	N
Call vendor?	N	N	N	Y	Y	Y	Y

Cause-Effect Graphs

- ▶ Cause-effect graphs are pictorial representations of the same business logic shown in a decision table.
- ▶ We can always convert one to the other.
- ▶ It is applied in the same way and to the same situations as a decision table.
- ▶ It requires tools to create the graphs, and you have to learn the notation.
- ▶ Cause-effect graph tests find the same types of bugs as decision tables.

Cause-Effect Graphs



Use Case Testing

- ▶ It is a way to ensure that we have tested **typical and exceptional workflows and scenarios for the system**, from the point of view of the various actors who directly interact with the system and from the point of view of the various stakeholders who indirectly interact with the system.
- ▶ It is important that the system is complete enough for the intended usage.
- ▶ The realistic usage of the system, including the atypical usages, must be well understood, or gaps will occur in the testing.
- ▶ Testing must cover the typical usage, the *main path* or *happy path*, as well as the atypical usages, called the *alternate paths* or *exception paths*.
- ▶ This technique can validate the software by checking for improper processing of main or alternate paths, missing alternate paths, and problems with error reporting.

Use Case Testing (2)

- ▶ To derive tests from Use Cases:
 - ▶ We should create a test for every workflow, including both the typical and exceptional workflows.
 - ▶ If the exceptional workflows were omitted,
 - ▶ then you'll need to figure those out from requirements or other sources.
 - ▶ Failing to test exceptions is a common testing mistake when using use cases.
 - ▶ Creating tests can involve applying equivalence partitioning and boundary value analysis along the way.

Example: Use Case

#	Test Step	Expected Result
1	Place one item in cart	Item in cart
2	Click check out	Checkout screen
3	Input valid US address, valid payment using American Express, and valid shipping method information	Each screen displays correctly, and valid inputs are accepted
4	Verify order information	Shown as entered
5	Confirm order	Order in system
6	Repeat steps 1–5, but place two items in cart, pay with Visa, and ship internationally	As shown in 1–5
7	Repeat steps 1–5, but place the maximum number of items in cart and pay with MasterCard	As shown in 1–5
8	Repeat steps 1–5, but pay with Discover	As shown in 1–5

exceptions



#	Test Step	Expected Result
1	Do not place any items in cart	Cart empty
2	Click check out	Error message
3	Place item in cart, click check out, enter invalid address, then invalid payment, then invalid shipping information	Error messages, can't proceed to next screen until resolved
4	Verify order information	Shown as entered
5	Confirm order	Order in system
6	Repeat steps 1–3, but stop activity and abandon transaction after placing item in cart	User logged out exactly ten minutes after last activity
7	Repeat steps 1–3, but stop activity and abandon transaction on each screen	As shown in 6
8	Repeat steps 1–4; do not confirm order	As shown in 6



Example: Use Case (2)

- ▶ We can easily translate a use case into one or more logical tests.
 - ▶ Translation of the logical tests into concrete tests can require additional documentation.
- ▶ For example, what was the maximum number of items we could put in the shopping cart?
 - ▶ We'd need some further information, ideally a requirements specification, to know that.
 - ▶ What items can we put in shopping cart? Some description of the store inventory is needed.

Example: Use Case

ID	02.001
Name	E-commerce Purchase
Actor	Customer
Description	Allow customer to complete a transaction by purchasing the item(s) in her shopping cart
Priority	Very high
Frequency of use	25% of customers, up to 1,000 customers per day
Preconditions	1. One or more items in shopping cart 2. Customer is logged in 3. Customer has clicked on check out

Typical workflow	1. System gathers address, payment, and shipping information from customer. 2. System displays all information for user confirmation. 3. User confirms order to system for delivery.
Exception 1	Customer attempts to check out with empty shopping cart. System gives error message.
Exception 2	Customer provides invalid address, payment, or shipping information. System gives error messages as appropriate.
Exception 3	Customer abandons transaction before or during checkout. System logs customer out after ten minutes of inactivity.
Postconditions	Order is active in system.



White-box (or code-based) test techniques

- ▶ Procedure to derive and/or select test cases based on an analysis of the internal structure of a component or system.
- ▶ Issue: in the normal activity, there is a really possibility that as much as 70 percent of all of the code that makes up the system might never have been executed once ...
 1. Condition Testing
 2. Decision Condition Testing
 3. Modified Condition/Decision Testing (MC/DC)
 4. Multiple Condition Testing
 5. Path Testing
 6. API Testing

Control Flow Testing Theory

- ▶ Control Flow Testing
 - ▶ An approach to structure-based testing in which test cases are designed to execute specific sequences of events.
- ▶ Control flow testing is done through control flow graphs, which provide a way to abstract a code module to better understand what it does.
- ▶ Control flow graphs give us a visual representation of the structure of the code.
- ▶ The algorithm for all control flow testing consists of converting a section of the code into a control graph and analyzing the possible paths through the graph.

How much should we test?

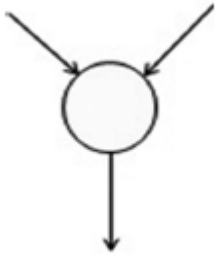
- ▶ Possible answers range from no testing at all to total exhaustive testing, hitting every possible path through the software.
- ▶ *Statement coverage.*
 - ▶ Have we exercised, at one time or another, every single line of executable code in the system? That would give us 100 percent statement coverage.
 - ▶ To calculate the total statement coverage actually achieved, divide the total number of executable statements into the number of statements that have actually been exercised by your testing.
- ▶ *Decision (or branch) coverage.*
 - ▶ This the total number of all of the outcomes of all of the decisions in the code that we have exercised
- ▶ *Condition coverage / multiple condition coverage*
 - ▶ we evaluate each atomic condition (all possible combinations) that makes up a decision at least once.

Building Control Flow Graphs

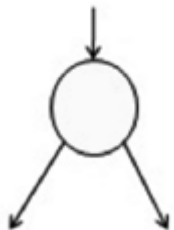


- ▶ The process block.
 - ▶ Graphically, it consists of a node (bubble or circle) with one path leading to it and one path leading from it.
 - ▶ This represents a chunk of code that executes sequentially: that is, no decisions are made inside of it.
 - ▶ The flow of execution reaches the process block, executes through that block of code in exactly the same way each time, and then exits, going elsewhere.
 - ▶ Decisions are the most significant part of the control flow concept; individual lines of code where no decisions are made do not affect the control flow and thus can be effectively ignored

Building Control Flow Graphs



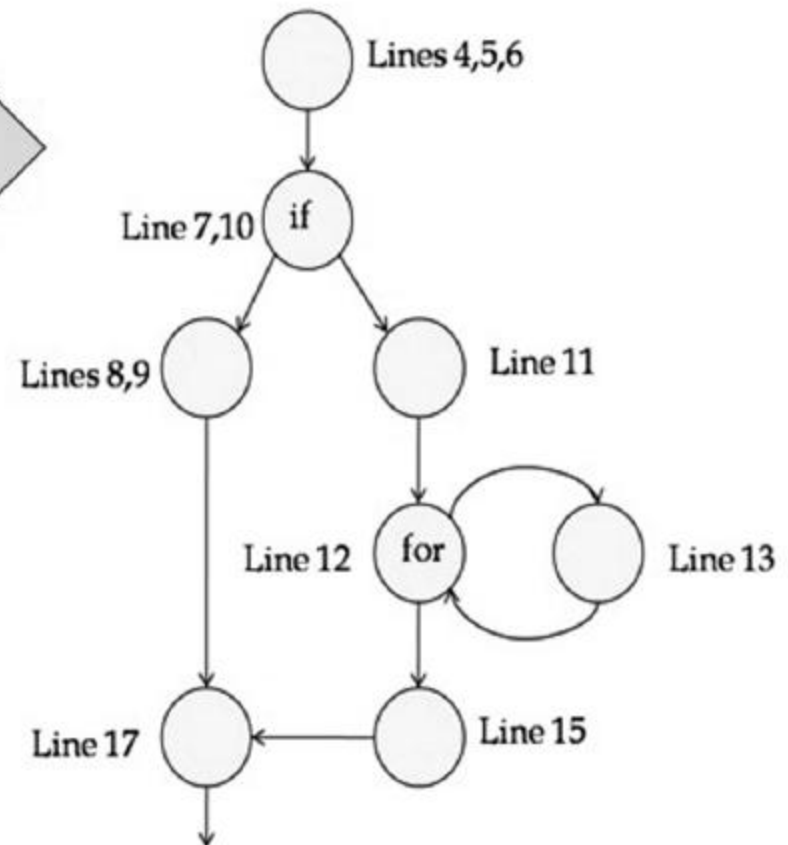
- ▶ junction point.
 - ▶ It may have any number of different paths leading into the process block, with only one path leading out.
 - ▶ No decisions are made in this block; all roads lead to it with only one road out.



- ▶ decision point.
 - ▶ It is represented as a node with one input and two or more possible outputs.
 - ▶ Decision points force us to have multiple tests, at least one test for each way to make the decision differently, changing the way we traverse the code.

Example

```
1 #include <stdio.h>
2 main()
3 {
4     int i, n, f;
5     printf("n = ");
6     scanf("%d", &n);
7     if (n < 0) {
8         printf("Invalid: %d\n", n);
9         n = -1;
10    } else {
11        f = 1;
12        for (i = 1; i <= n; i++) {
13            f *= i;
14        }
15        printf("%d! = %d\n", n, f);
16    }
17    return n;
18 }
```



Statement Coverage

- ▶ Statement-level coverage is considered the least effective of all control flow techniques.
 - ▶ In order to reach this modest level of coverage, a tester must come up with enough test cases to force every line to execute at least once.
 - ▶ Achieving (at least) statement coverage can (and should) be done while unit testing.
- ▶ The Institute of Electrical and Electronics Engineers (IEEE), in the unit test standard ANSI 87B (1987), stated that statement coverage was the minimum level of coverage that should be acceptable.
- ▶ Boris Beizer, in his seminal work, *Software Testing Techniques* (ITC Press 1990), has a slightly more inflammatory take on it: “... testing less than this for new software is unconscionable and should be criminalized.”

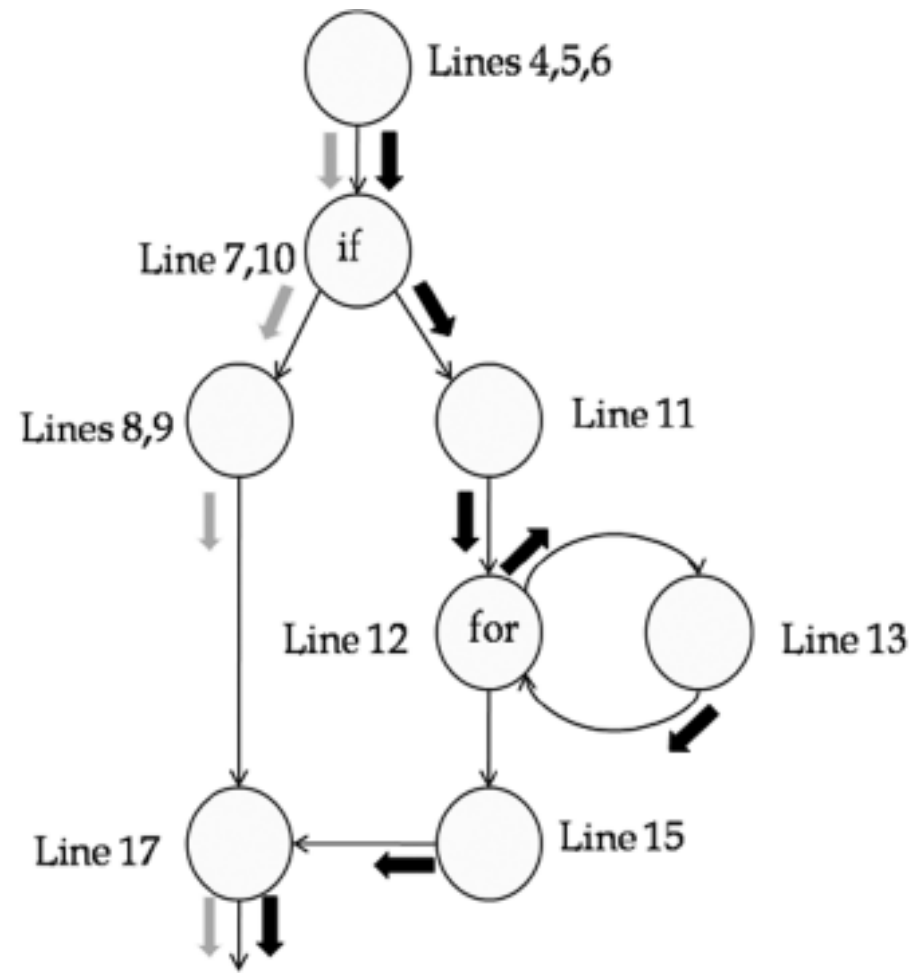
Beizer rules

► Beizer lays out some rules of common sense:

1. *Not testing a piece of code leaves a residue of bugs in the program in proportion to the size of the untested code and the probability of bugs.*
2. *The high-probability paths are always thoroughly tested if only to demonstrate that the system works properly. If you have to leave some code untested at the unit level, it is more rational to leave the normal, high-probability paths untested, because someone else is sure to exercise them during integration testing or system testing.*
3. *Logic errors and fuzzy thinking are inversely proportional to the probability of the path's execution.*
4. *The subjective probability of executing a path as seen by the routine's designer and its objective execution probability are far apart. Only analysis can reveal the probability of a path, and most programmers' intuition with regard to path probabilities is miserable.*
5. *The subjective evaluation of the importance of a code segment as judged by its programmer is biased by aesthetic sense, ego, and familiarity. Elegant code might be heavily tested to demonstrate its elegance or to defend the concept, whereas straightforward code might be given cursory testing because "How could anything go wrong with that?"*

Statement Coverage example

```
1 #include <stdio.h>
2 main()
3 {
4     int i, n, f;
5     printf("n = ");
6     scanf("%d", &n);
7     if (n < 0) {
8         printf("Invalid: %d\n", n);
9         n = -1;
10    } else {
11        f = 1;
12        for (i = 1; i <= n; i++) {
13            f *= i;
14        }
15        printf("%d! = %d\n", n, f);
16    }
17    return n;
18 }
```



Statement Coverage failure

1. $z = 0;$
2. if $(a > b)$ then
3. $z = 12;$
4. $Rep = 72/z;$

Test 1: $a = 3, b = 2 \rightarrow Rep = 6$

Decision Coverage

- ▶ This level of coverage looks at the decisions themselves.
 - ▶ Every decision has the possibility of being resolved as either TRUE or FALSE.
- ▶ For those who point out that the switch statement can make more than two decisions, well, conceptually that appears to be true.
 - ▶ Each atomic decision in the switch statement is a comparison between two values TRUE or FALSE.
- ▶ To get to the 100 percent decision level of coverage, every decision made by the code must be resolved both ways at one time or another, TRUE and FALSE.
- ▶ Note that if we execute each decision both ways, giving us decision coverage, it guarantees statement coverage in the same code.
 - ▶ Therefore, decision coverage is said to be stronger than statement coverage.
- ▶ We calculate the exact level of decision coverage by dividing the number of decision outcomes tested by the total number of decision outcomes in the module/system.

Typical language decision constructs

Either/or Decisions	Loop Decisions
<pre>if (expr) {} else {}</pre>	<pre>while (expr) {}</pre>
<pre>switch (expr) { case const_1: {} break; case const_2: {} break; case const_3: {} break; case const_4: {} break; default {} }</pre>	<pre>do {} while (expr)</pre> <pre>for (expr_1; expr_2; expr_3) {}</pre>

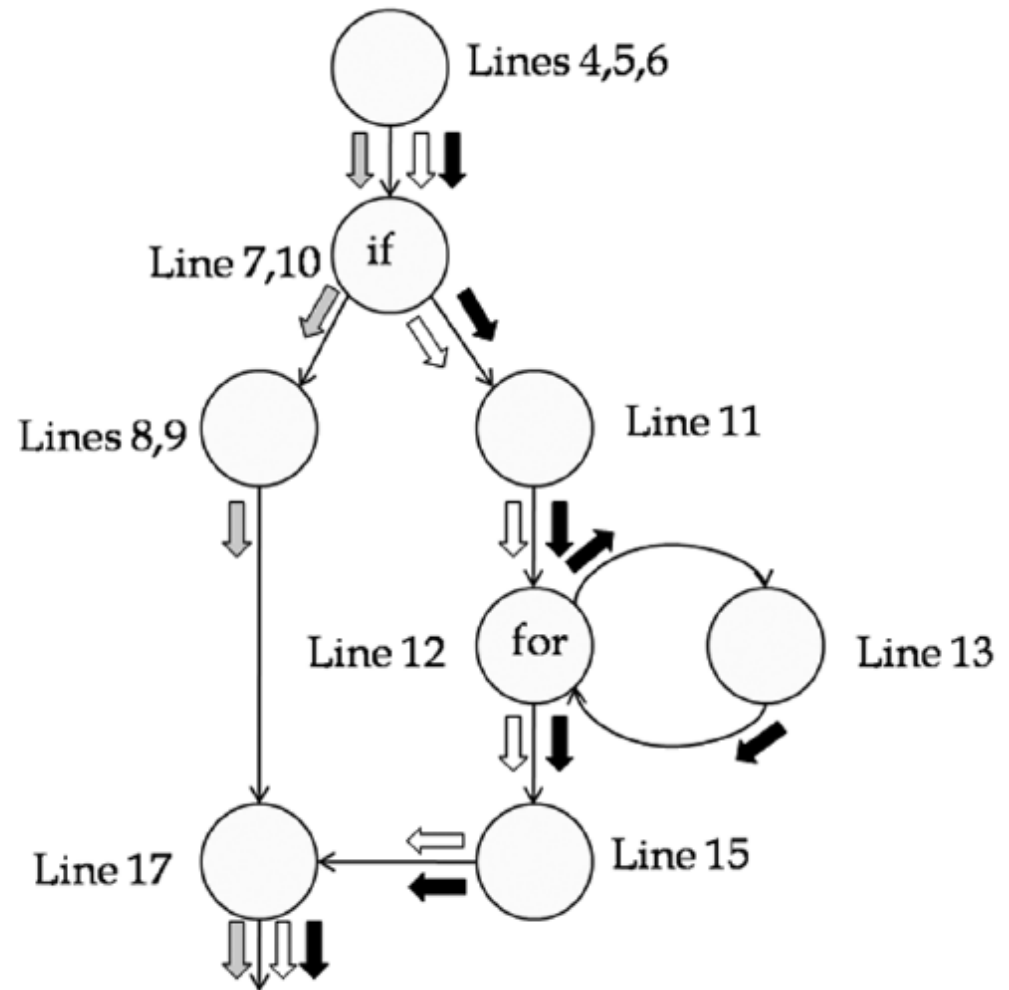
Decision coverage example

```

1 #include <stdio.h>
2 main()
3 {
4   int i, n, f;
5   printf("n = ");
6   scanf("%d", &n);
7   if (n < 0) {
8     printf("Invalid: %d\n", n);
9     n = -1;
10  } else {
11    f = 1;
12    for (i = 1; i <= n; i++) {
13      f *= i;
14    }
15    printf("%d! = %d\n", n, f);
16  }
17  return n;
18 }

```

Statement coverage: We tested
($n < 0$) and ($n > 0$)



Loop Coverage

- ▶ If we want to completely test a loop, we would need to test it zero times (i.e., did not loop), one time, two times, three times, all the way up to n times where it hits the maximum it will ever loop.
- ▶ Different theories have been offered that try to prescribe how much testing we should give a loop.
- ▶ The basic minimum tends to be two tests, zero times through the loop and one time through the loop (giving us decision coverage).
- ▶ Others add a third test, multiple times through the loop, although they do not specify how many times.
- ▶ We suggest that you try to test the loop zero and one time and then test the maximum number of times it is expected to cycle (if you know how many times that is likely to be).
- ▶ Our bug hypothesis is pretty clear; failing to test the loop (especially when it does not loop even once) may cause bugs to be shipped to production.

Loop Coverage example

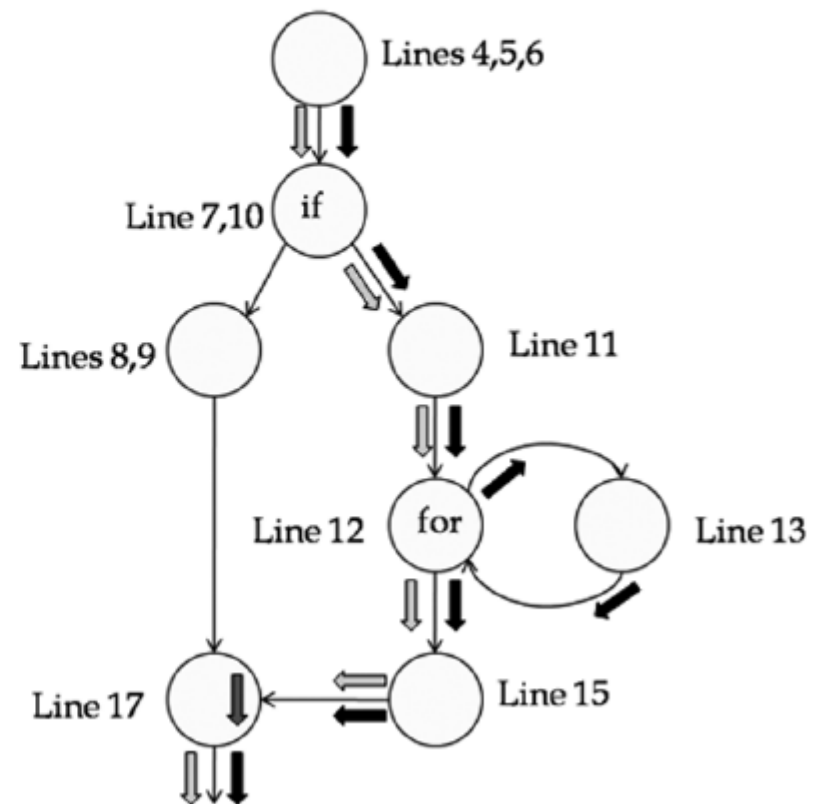
```

1 #include <stdio.h>
2 main()
3 {
4     int i, n, f;
5     printf("n = ");
6     scanf("%d", &n);
7     if (n < 0) {
8         printf("Invalid: %d\n", n);
9         n = -1;
10    } else {
11        f = 1;
12        for (i = 1; i <= n; i++) {
13            f *= i;
14        }
15        printf("%d! = %d\n", n, f);
16    }
17    return n;
18 }

```

Loop 0 times by testing ($n=0$)

Loop 1 time by testing ($n = 1$)

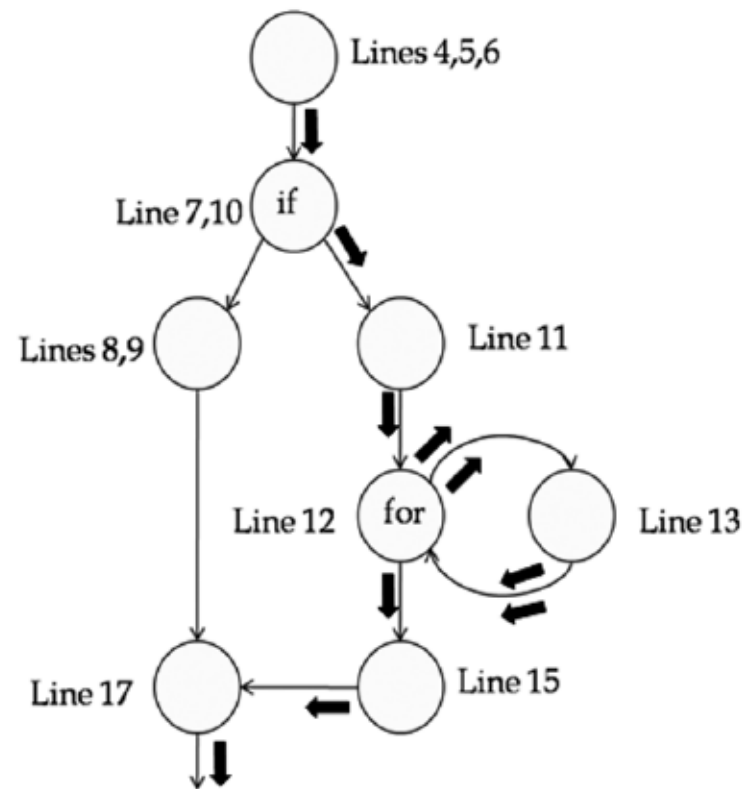


Loop Coverage example

```
1 #include <stdio.h>
2 main()
3 {
4     int i, n, f;
5     printf("n = ");
6     scanf("%d", &n);
7     if (n < 0) {
8         printf("Invalid: %d\n", n);
9         n = -1;
10    } else {
11        f = 1;
12        for (i = 1; i <= n; i++) {
13            f *= i;
14        }
15        printf("%d! = %d\n", n, f);
16    }
17    return n;
18 }
```

Loop 12 times by testing ($n==12$)

Loop 13 times and it overflows



Boris Beizer Rules

- ▶ If possible, test a value that is one less than the expected minimum value the loop can take. For example, if we expect to loop with the control variable going from 0 to 100, try -1 and see what happens.
- ▶ Try the minimum number of iterations—usually zero iterations. Occasionally, there will be a positive number as the minimum.
- ▶ Try one more than the minimum number.
- ▶ Try to loop once (this test case may be redundant and should be omitted if it is).
- ▶ Try to loop twice (this might also be redundant).
- ▶ A typical value. Beizer always believes in testing what he often calls a nominal value. Note that the number of loops, from one to max, is actually an equivalence set. The nominal value is often an extra test that we tend to leave out.
- ▶ One less than the maximum value.
- ▶ The maximum number of values.
- ▶ One more than the maximum value.

Nested loops

1. Starting at the innermost loop, set all outer loops to minimum iteration setting.
2. Test the boundary values for the innermost loop as discussed earlier.
3. If you have done the outermost loop already, go to step 5.
4. Continue outward, one loop at a time until you have tested all loops.
5. Test the boundaries of all loops simultaneously. That is, set all to 0 loops, 1 loop, maximum loops, 1 more than maximum loops.

Condition Coverage

- ▶ A white-box test design technique in which test cases are designed to execute condition outcomes
 - ▶ The basic concept is that, when a decision is made by a complex expression that evaluates to TRUE or FALSE, each atomic condition is tested both ways, TRUE and FALSE.
 - ▶ **atomic condition:** A condition that cannot be decomposed, i.e., a condition that does not contain two or more single conditions joined by a logical operator (AND, OR, XOR).

$(A \parallel B) \&\& (C == D)$

$(a > b) \parallel (x + y == -1) \&\& ((d) != \text{TRUE})$

Example: 2 examples with 3 atomic conditions

- ▶ Decision and condition coverage will always be exactly the same when all decisions are made by simple atomic expressions.
 - ▶ For the conditional, *if* $(a == 1) \{ \}$, condition coverage will be identical to decision coverage.

Condition Coverage: example

if (A && B) then

{Do something}

else

{Do something else}

Test 1: $A == \text{FALSE}$, $B == \text{TRUE}$ resolves to FALSE

Test 2: $A == \text{TRUE}$, $B == \text{FALSE}$ resolves to FALSE

- ▶ Do we now have condition coverage?
 - ▶ A was set both TRUE and FALSE , as was B . Sure, we have achieved condition coverage.
- ▶ But do we have decision coverage?
 - ▶ The answer is no!
- ▶ Condition coverage is *not* stronger than decision coverage because 100 percent condition coverage does not imply 100 percent decision coverage.
- ▶ That makes condition coverage—by itself—not very interesting for structural testing.
 - ▶ However, it is still an important test design technique; we just need to extend the technique a bit to make it stronger.

Decision Condition Coverage

- ▶ white-box test design technique in which test cases are designed to execute condition outcomes and decision outcomes
- ▶ This level of coverage is just a combination of decision and condition coverage.
 - ▶ We need to achieve **condition level coverage** where each atomic condition is tested both ways, TRUE and FALSE, *and* we also need to make sure that we achieve **decision coverage** by ensuring that the overall predicate be tested both ways, TRUE and FALSE.
- ▶ The bug hypothesis should be clear from the preceding discussion; not testing for decision coverage even if we have condition coverage may allow bugs to remain in the code.
- ▶ Full decision condition coverage guarantees condition, decision, and statement coverage and thereby is stronger than all three.

Decision Condition Coverage: example

- ▶ (A && B)
 - ▶ A is set to FALSE and B is set to TRUE.
 - ▶ A is set to TRUE and B is set to FALSE.
- ▶ We can add a third test case:
 - ▶ Both A and B are set to TRUE.
- ▶ An alternative would be:
 - ▶ A and B are both set to TRUE.
 - ▶ A and B are both set to FALSE.
- ▶ In this case, we can achieve full decision condition coverage with only two test cases.

Modified Condition/Decision Coverage

- ▶ Like decision condition coverage, MC/DC requires that each atomic condition be tested both ways and that decision coverage must be satisfied. It then adds two more constraints:
 1. For each atomic condition, there is at least one test case where the entire predicate evaluates to FALSE only because that specific atomic condition evaluated to FALSE.
 2. For each atomic condition, there is at least one test case where the entire predicate evaluates to TRUE only because that specific atomic condition evaluated to TRUE
- ▶ In most cases, this level of coverage requires $(N + 1)$ test cases
 - ▶ (N) is the total number of atomic conditions in the predicate.

Modified Condition/Decision Coverage (2)

- ▶ Many references use the term *MC/DC* as a synonym for exhaustive testing.
 - ▶ However, this level of testing does not rise to the level of exhaustive testing,
 - ▶ This would include other testing (running every loop every possible number of times, for example).
 - ▶ *MC/DC* does, however, exercise each complex predicate in a number of ways that should result in finding certain types of subtle defects.
- ▶ *MC/DC* coverage may not be useful to all testers.
 - ▶ It will likely be most useful to those who are testing mission- or safety-critical software.
- ▶ According to the most prevalent legend, it was originally created at Boeing to be used in safety-critical software development.
- ▶ There are several algorithms advanced to define the test cases needed to achieve *MC/DC*-level coverage.

MC/DC-level coverage

- ▶ A neutral value for an atomic condition is the one that has no effect on the eventual outcome evaluation of the expression.
 - ▶ The operator used in joining two atomic conditions determines exactly what value will be neutral.
- ▶ **A AND B**
 - ▶ The neutral value will be TRUE.
 - ▶ If we set *B* to the neutral value, we control the output by toggling *A*.
 - ▶ $(A \text{ AND } B) \rightarrow (\text{TRUE AND } \textit{TRUE} == \text{TRUE}), (\text{FALSE AND } \textit{TRUE} == \text{FALSE})$
- ▶ **A OR B**
 - ▶ The neutral value must evaluate to FALSE.
 - ▶ $(A \text{ OR } B) \rightarrow (\text{TRUE OR } \textit{FALSE} == \text{TRUE}), (\text{FALSE OR } \textit{FALSE} == \text{FALSE})$

Example

- ▶ (A OR B) AND (C AND D)

(A OR B) AND (C AND D)	T	F
A	T -- --	F -- --
B	-- T --	-- F --
C	-- -- T --	-- -- F --
D	-- -- -- T	-- -- -- F

This means that we have to test with True Neutral Neutral Neutral

Example

► (A OR B) AND (C AND D)

(A OR B) AND (C AND D)	T	F
A	T F T T	F F T T
B	F T T T	F F T T
C	T F T T	T T F T
D	T F T T	T T T F

(A OR B) AND (C AND D)	T	F
A	T F T T	F F T T
B	F T T T	F F T T
C	T F T T	T T F T
D	T F T T	T T T F

Complicating Issues

Short-Circuiting

- ▶ In some programming languages Boolean expressions may be resolved without evaluating every subexpression, depending on the Boolean operator that is used.

Value of A	Value of B	A B
T	T	T
T	F	T
F	T	T
F	F	F

If A is TRUE, then it does not matter what the value of B is

We are not evaluating all conditions

Complicating Issues Coupling

- ▶ This happens when there are multiple occurrences of a condition in an expression.
 - ▶ $A \parallel (!A \ \&\& \ B)$
- ▶ A and $!A$ coupled.
 - ▶ They cannot be varied independently as the MC/DC coverage rule says they must.
 - ▶ So how does a test analyst actually need to test this predicate and still achieve the required coverage?
- ▶ There are different techniques out of the scope of this course.

Multiple Condition Coverage

- ▶ The concept is to test every possible combination of atomic conditions in a decision.
 - ▶ This is an exhaustive testing.
- ▶ Create a truth table with every combination of atomic conditions, and come up with values that test each one.
 - ▶ The number of tests is proportional to the number of atomic conditions, using the formula 2^n , where n is the number of atomic conditions (assuming no short-circuiting).
- ▶ The bug hypothesis is bugs could be anywhere, lurking in some esoteric combinations of atomic conditions that are possible but unlikely!
- ▶ We can measure the amount of multiple condition coverage by calculating the theoretical possible numbers of permutations we could have and dividing those into the number that we actually test.
 - ▶ Note that short-circuit can affect the number of tests to execute

Path Testing

- ▶ We have covered three main ways of approaching white-box test design so far:
 1. Statement testing where the statements themselves drove the coverage
 2. Decision/branch testing where the decision predicate drove the coverage
 3. Condition, decision condition, modified condition/decision coverage, and multiple condition coverage, all of which looked at subexpressions and atomic conditions of a particular decision predicate to drive the coverage
- ▶ Rather than concentrating on control flows, path testing is going to identify interesting paths through the code that we want to test.
- ▶ In some cases we will get some of the same tests we got through control flow testing.
- ▶ But we might come up with some other interesting tests.

Path Testing via Flow Graphs

- ▶ The first step in this path coverage technique is to create a control flow graph for the code we want to test.
 1. *Pick the simplest, functionally sensible entry/exit path.*
 2. *Pick additional paths as small variations from previous paths. Pick paths that do not have loops rather than paths that do. Favor short paths over long paths, simple paths over complicated paths, and paths that make sense over paths that don't.*
 3. *Pick additional paths that have no obvious functional meaning only if it's necessary to provide coverage. But ask yourself first why such paths exist at all. Why wasn't coverage achieved with functionally sensible paths?*
 4. *Be comfortable with your chosen paths. Play your hunches and give your intuition free reign as long as you achieve C1 + C2. (100 percent statement and branch coverage)*
 5. *Don't follow rules slavishly—except for coverage*

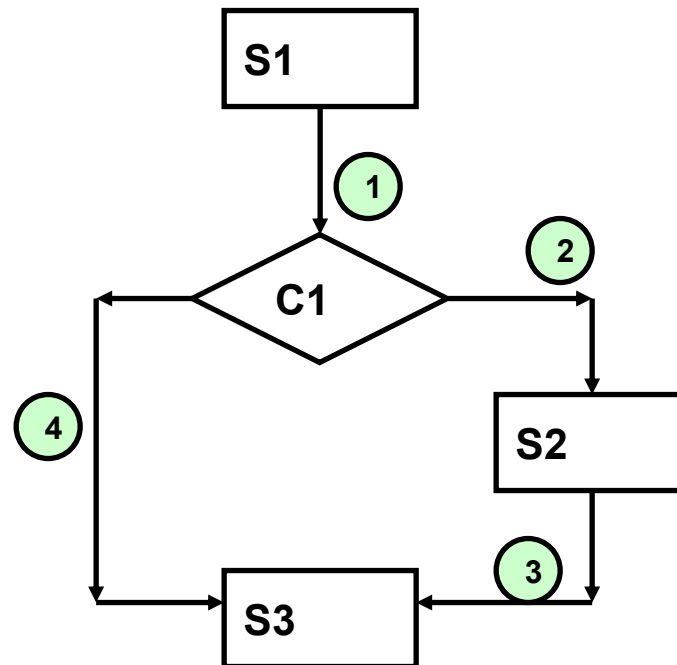
Basis Path Testing

- ▶ McCabe theorized that any software module has a small number of unique, independent paths through it (if we exclude iterations).
 - ▶ He called these *basis paths*.
- ▶ His theory suggests that the structure of the code can be tested by executing through this small number of paths and that all the infinity of different paths actually consists of simply using and reusing the basis paths.

Path Analysis

- ▶ Path analysis provides a test design technique that is reproducible, traceable, and countable.
 - ▶ It is often used as a white-box testing technique
- ▶ Two tasks are involved in path analysis:
 1. Analyze the number of paths that exist in the system or program.
 2. Determine how many paths should be included in the test.
- ▶ Testing the linear independent paths
 - ▶ a linearly independent path is any path through the application that introduces at least one new edge that is not included in any other linearly independent path

Path Analysis (2)



Path1 : S1 – C1 – S3

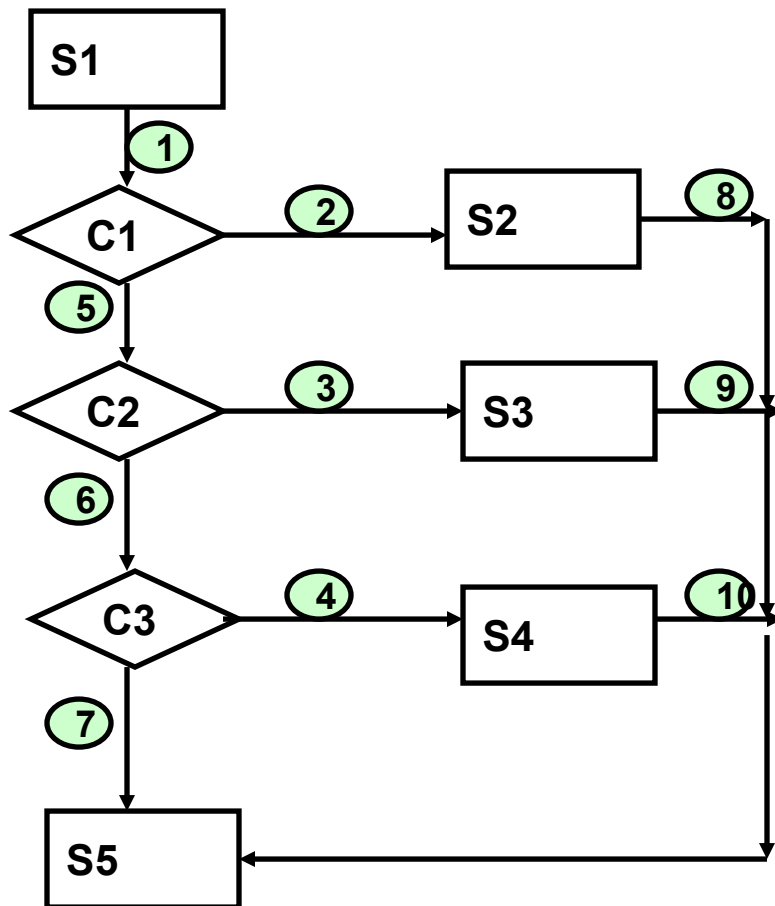
Path2 : S1 – C1 – S2 – S3

OR

Path1: segments (1,4)

Path2: segments (1,2,3)

Path Analysis (3)



The 4 Independent Paths Covers:

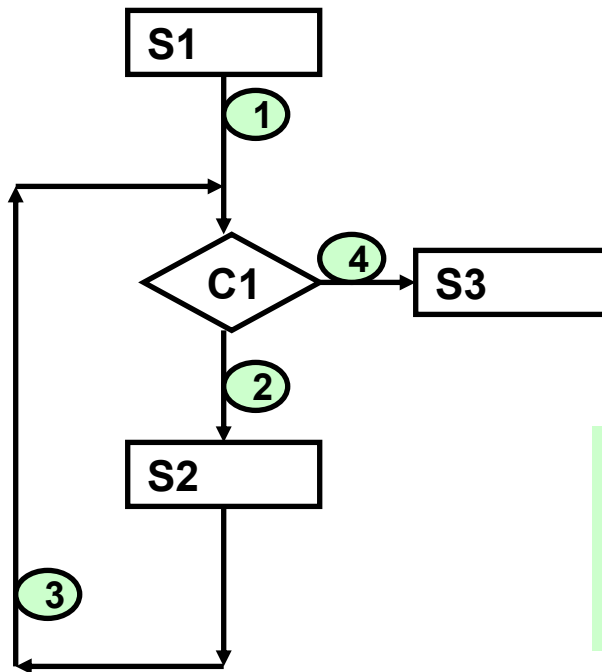
Path1: includes S1-C1-S2-S5

Path2: includes S1-C1-C2-S3-S5

Path3: includes S1-C1-C2-C3-S4-S5

Path4: includes S1-C1-C2-C3-S5

Path Analysis (4)



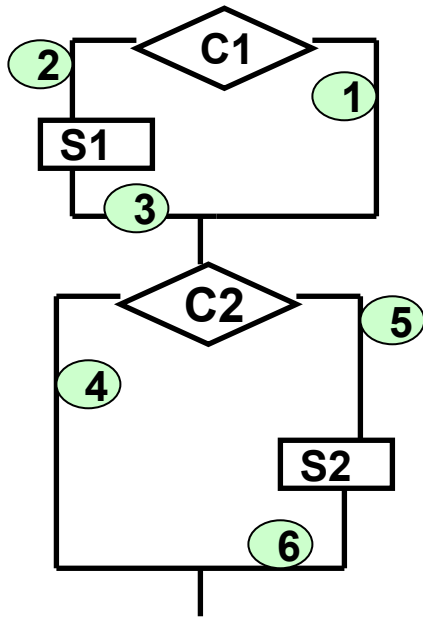
Linearly Independent Paths are:

path1 : S1-C1-S3 (segments 1,4)

path2 : S1-C1-S2-C1-S3 (segments 1,2,3,4)

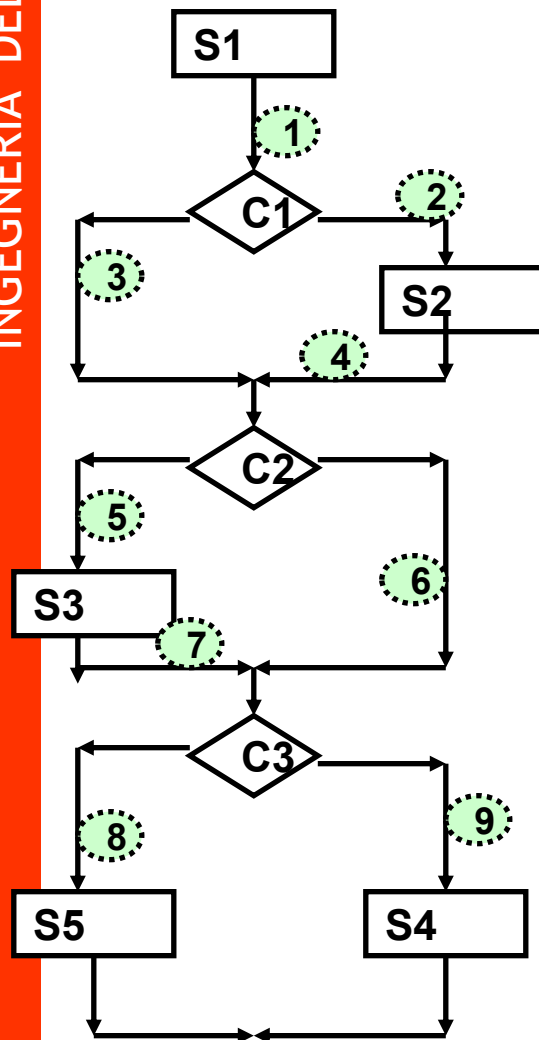
Path Analysis (5)

Consider path1, path2 and path3 as the Linearly Independent Set



	1	2	3	4	5	6
path1	1				1	1
path2	1			1		
path3		1	1	1		
path4		1	1		1	1

Path Analysis (6)



Since for each binary decision, there are 2 paths and there are 3 in sequence, there are $2^3 = 8$ total “logical” paths

path1 : S1-C1-S2-C2-C3-S4
 path2 : S1-C1-S2-C2-C3-S5
 path3 : S1-C1-S2-C2-S3-C3-S4
 path4 : S1-C1-S2-C2-S3-C3-S5

path5 : S1-C1-C2-C3-S4
 path6 : S1-C1-C2-C3-S5
 path7 : S1-C1-C2-S3-C3-S4
 path8 : S1-C1-C2-S3-C3-S5

How many Linearly Independent paths are there?
 Using **Cyclomatic number** = 3 decisions +1 = 4

One set would be:

path1 : includes segments (1,2,4,6,9)
 path2 : includes segments (1,2,4,6,8)
 path3 : includes segments (1,2,4,5,7,9)
 path5 : includes segments (1,3,6,9)

Experience-based Testing

- ▶ It uses the users' and the testers' experience
 - ▶ to determine the most important areas of a system
 - ▶ to exercise these areas in ways that are both consistent with expected use and likely to be the sites of errors - this is where the experience comes in
- ▶ **Error guessing**
 - ▶ It takes advantage of a tester's skill, intuition and experience with similar applications to identify special tests that may not be easy to capture by the more formal techniques.
 - ▶ It can add value in identifying test cases that
 - ▶ target suspected weaknesses or
 - ▶ address aspects of the application that have been found to be problematical in the past.
 - ▶ The main drawback is its varying effectiveness, depending as it does on the experience of the tester deploying it.
 - ▶ If several testers and/or users contribute to constructing a list of possible errors and tests are designed to attack each error listed, this weakness can be effectively overcome.
 - ▶ Another way to make error guessing more structured is by starting from the creation of defect and failure lists.

Experience-based Testing

▶ Exploratory testing

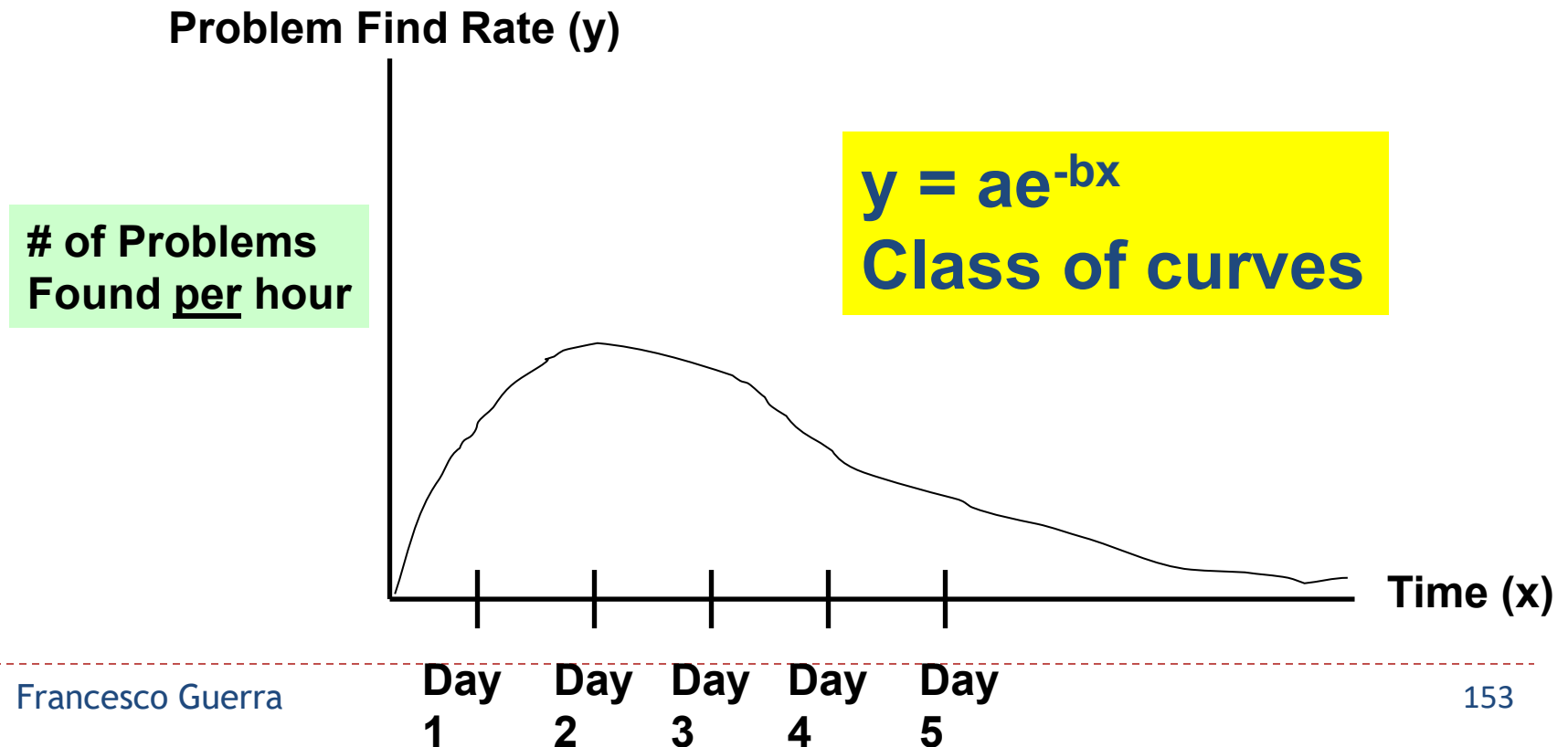
- ▶ It combines the experience of testers with a structured approach to testing where specifications are either missing or inadequate and where there is severe time pressure.
- ▶ It exploits concurrent test design, test execution, test logging and learning within time-boxes and is structured around a test charter containing test objectives.
- ▶ In this way exploratory testing maximises the amount of testing that can be achieved within a limited time frame, using test objectives to maintain focus on the most important areas.

▶ Checklist-based testing

- ▶ It is testing based on high-level checklists, which may be drawn from many sources and generally encapsulate individuals' experience and information gleaned from other sources, such as standards, known problem areas, expected use scenarios and any other relevant sources. Checklists are a guide to the testing required, so are used mainly by experienced testers as a source of ideas from which detailed tests are derived.
- ▶ The checklist is drawn up during test analysis as a set of test conditions and may reuse or update previous checklists
 - ▶ Coverage is defined by completion of the checklist and may be higher than for other forms of testing, though repeatability will typically be lower than for more formal testing.

When to Stop Testing

- ▶ A simple answer is to stop testing when all the planned test cases are executed and all the problems found are fixed.
 - ▶ In reality, we are often pressured by schedule to release software product.
- ▶ Keeping track of the test results and observing statistics.



FindBugs (now SpotBugs)

- ▶ FindBugs, a program which uses static analysis to look for bugs in Java code.
- ▶ FindBugs recognizes more than 300 programming mistakes and dubious coding idioms that it can identify using simple analysis techniques.
 - ▶ FindBugs also uses more sophisticated analysis techniques, devised to help effectively identify certain issues—such as dereferencing of null pointers— that occur frequently enough to warrant their development.
- ▶ Many developers use FindBugs ad hoc, and a growing number of projects and companies are integrating it into their standard build and testing systems.
 - ▶ Google has incorporated FindBugs into its standard testing and code-review process and has fixed more than 1,000 issues in its internal code base that FindBugs has identified.

Examples of detected bugs

```
public String foundType() {  
    return this.foundType();  
}
```

- ▶ This code should have been a getter method for the field `foundType`, but the extra parenthesis means it always recursively calls itself until the stack overflows.
- ▶ Another common bug pattern is when software invokes a method but ignores its return value, despite the fact that doing so makes no sense.
 - ▶ `s.toLowerCase()`, where `s` is a `String`
- ▶ Because `Strings` in Java are immutable, the `toLowerCase()` method has no effect on the `String` it's invoked on, but rather returns a new `String`.
 - ▶ `s = s.toLowerCase()`.

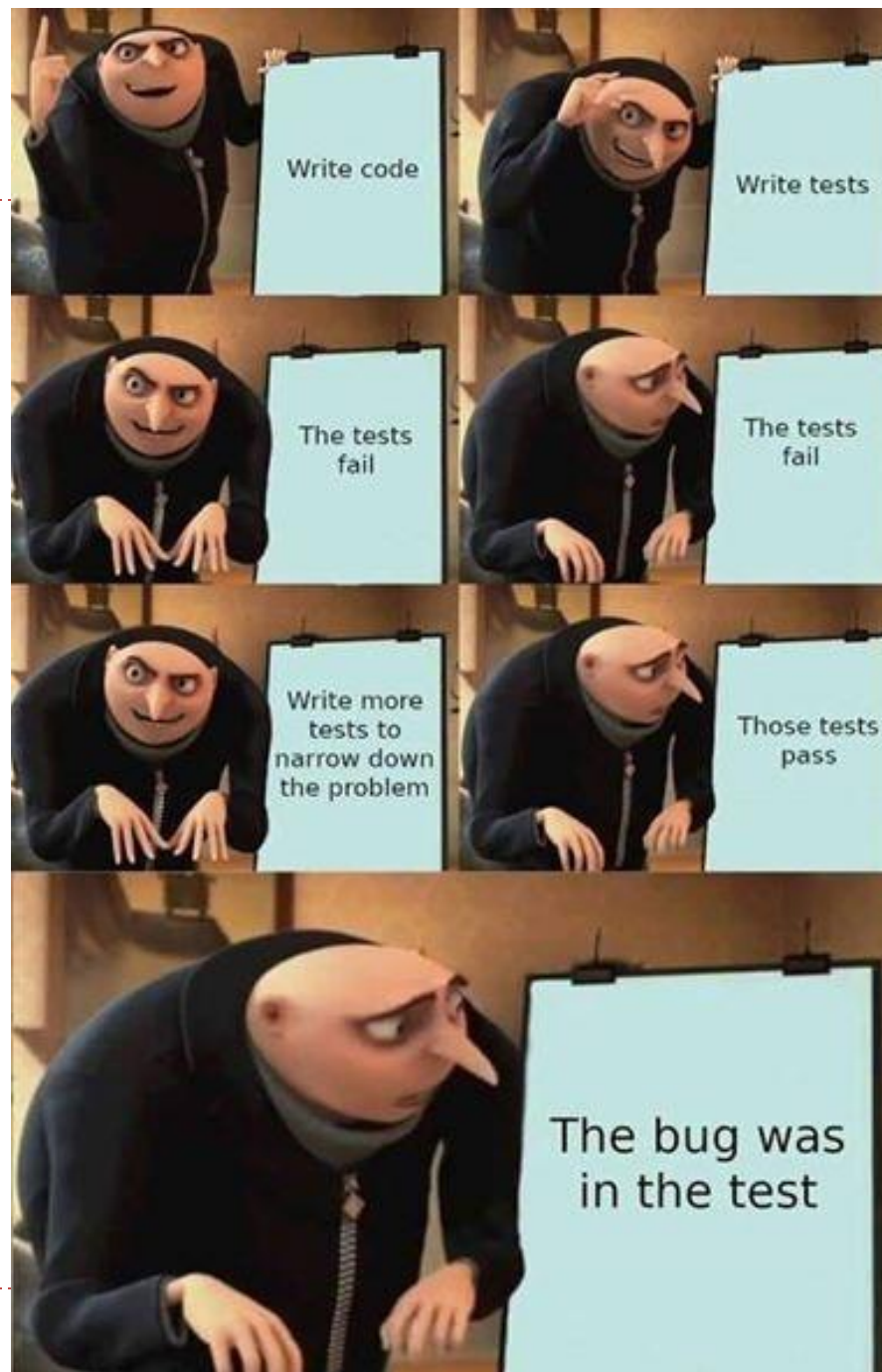
Examples of detected bugs (2)

- ▶ One common case is using the wrong relational or Boolean operation
(`name != null || name.length > 0`).
- ▶ Java evaluates the `&&` and `||` operators using short-circuit evaluation:
 - ▶ the right-hand side is evaluated only if needed in order to determine the expression's value.
- ▶ Java will evaluate the expression `name.length` only when `name` is null, leading to a null pointer exception.

Table I

**Users that review at least high-priority warnings
for each category (out of 252)**

Bug category reviewed	Percentage of users
Bad practice	96
Performance	96
Correctness	95
Multithreaded correctness	93
Malicious code vulnerability	86
Dodgy	86
Internationalization	57





References

- ▶ Frank Tsui, Orlando Karam, Barbara Bernal, Software engineering (Third Edition), Jones & Bartlett Learning
- ▶ http://www.tutorialspoint.com/software_testing
- ▶ Rex Black: Advanced Software Testing - Vol. 1, **2nd Edition** : Guide to the ISTQB Advanced Certification as an Advanced Test Analyst (Rockynook Computing), January 2016
- Rex Black: Advanced Software Testing - Vol. 3: Guide to the ISTQB Advanced Certification as an Advanced Test Analyst(Rockynook Computing), January 2015
- Paul Ammann, Jeff Offutt: Introduction to Software Testing, Cambridge University Press 2017
- Brian Hambling Software Testing: An ISTQB-BCS Certified Tester Foundation Guide, Fourth Edition BCS © 2019

MATERIALE DIDATTICO INTERNO DEL CORSO DI Ingegneria del Software.

La diffusione e pubblicazione on line del materiale è assolutamente proibita.